

IncidentFlow

The complete end-to-end guide to building this project from scratch

Every command, every file, every design decision
Generated from a full analysis of the repository — August 2026

Table of contents

1. What IncidentFlow is
2. The architecture in one picture
3. Division of responsibility
4. Prerequisites
5. Scaffolding the repository
6. Part 1 — Laravel API (backend)
 - 6.1 Creating the project & dependencies
 - 6.2 Environment files
 - 6.3 Configuration (config/)
 - 6.4 bootstrap/app.php — the wiring
 - 6.5 HTTP middleware (6 classes)
 - 6.6 Enums — the domain vocabulary
 - 6.7 Migrations — the schema
 - 6.8 Models
 - 6.9 Services — the business logic
 - 6.10 Authentication (JWT guard, tokens)
 - 6.11 Controllers, Requests, Resources
 - 6.12 Policies — authorization
 - 6.13 Jobs, Mail and the notification pipeline
 - 6.14 Console commands & the scheduler
 - 6.15 Exceptions — one error shape
 - 6.16 Routes
 - 6.17 Seeders & factories
 - 6.18 Tests (110 tests)
 - 6.19 The API Docker image
7. Part 2 — Realtime service (Node + Express)
 - 7.1 Project setup
 - 7.2 config.ts — the env contract
 - 7.3 redis.ts — the Redis subscriber
 - 7.4 hub.ts — channel hub & replay
 - 7.5 transports — SSE and WebSocket
 - 7.6 auth/jwt.ts — ticket verification
 - 7.7 app.ts, index.ts, middleware, metrics, logger
 - 7.8 Tests and Docker image
8. Part 3 — Web SPA (React 19 + Vite)
 - 8.1 Scaffold & dependencies
 - 8.2 vite.config.ts and TypeScript
 - 8.3 lib/ — the door to the API
 - 8.4 auth/ — session management
 - 8.5 realtime/ — the live stream hook
 - 8.6 hooks/ — React Query data
 - 8.7 components/
 - 8.8 pages/
 - 8.9 App, routing, styling
 - 8.10 Unit tests, e2e tests, Docker image
9. Part 4 — infra/nginx (the edge)
10. Part 5 — Docker Compose (dev + prod)
11. Part 6 — GitHub Actions CI/CD
12. Part 7 — Makefile & documentation
13. Running the whole stack
14. Demo accounts & what is implemented
15. Verifying everything (tests)
16. Operating it — the runbook essentials
17. Appendix A — the complete command log
18. Appendix B — file inventory

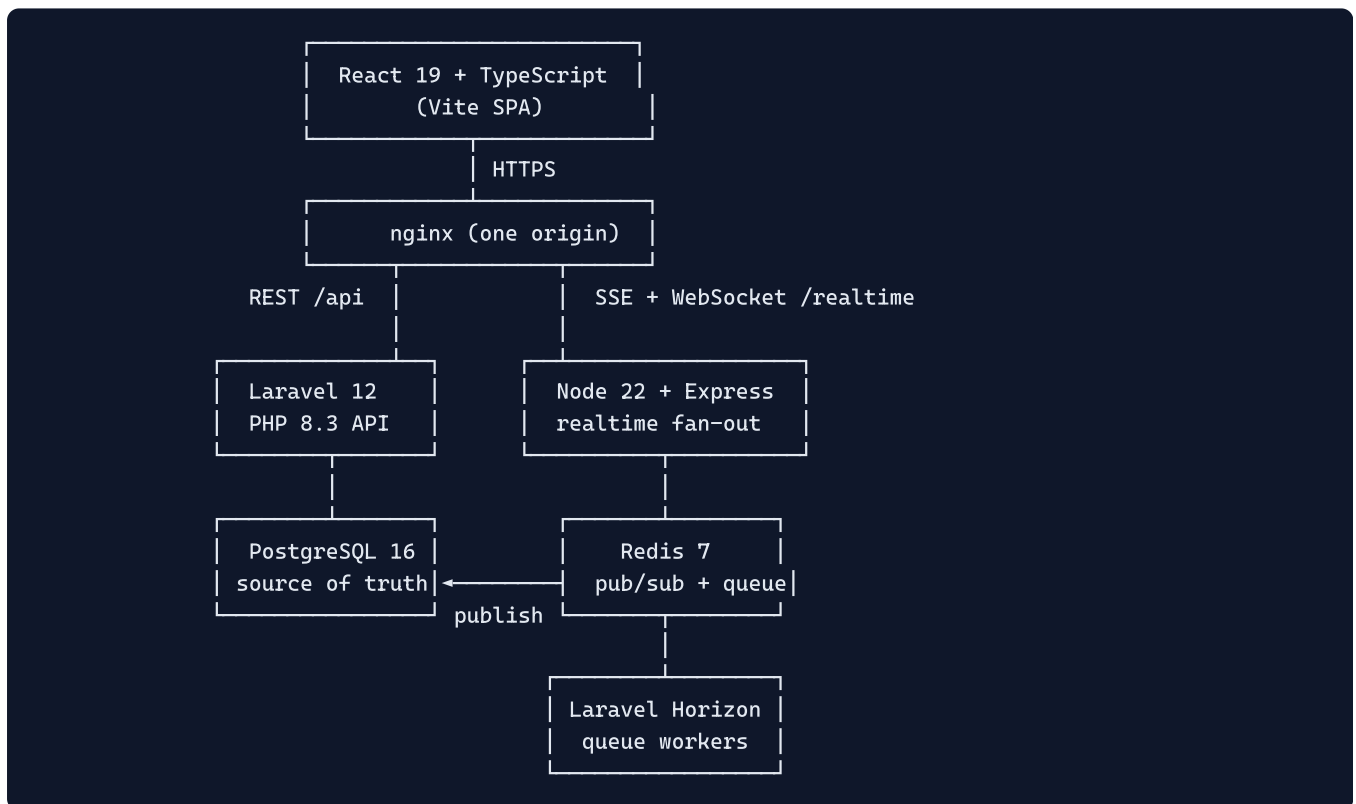
1. What IncidentFlow is

IncidentFlow is a **production-grade incident management platform**: employees report production incidents, engineers coordinate the response, and managers track service reliability. It was built as a **modular service-oriented architecture** — deliberately *not* microservices — with:

- **One database of record** (PostgreSQL 16) that owns every fact the product asserts.
- **One message bus** (Redis 7) carrying events between services.
- **Two backend services** with clearly separated responsibilities: a Laravel 12 REST API (PHP 8.3) and a Node 22 + Express realtime fan-out service.
- **One single-page frontend** (React 19 + TypeScript + Vite) served behind nginx as a single origin.

Core features: registration/login, incident creation with SEV-1...SEV-4 severity, responder assignment, a guarded status lifecycle (open → acknowledged → mitigated → resolved → closed), an append-only timeline, internal comments, live updates over SSE/WebSocket, postmortems, MTTA/MTTR metrics with percentiles, a permanent audit log, queued email, CSV export, and an admin view. Plus production concerns: RBAC, validation, rate limiting, idempotency keys, pagination, optimistic UI, structured JSON logs, correlation IDs across all three services, health probes, soft deletes, transactions, queue retries with backoff, OpenAPI docs, four test suites, dev/prod Docker configs, GitHub Actions CI with image scanning — and seeded demo data.

2. The architecture in one picture



3. Division of responsibility

Service	Owns	Never does
Laravel API	Persistent business state, authorization, the incident lifecycle, the audit trail	Hold open connections
Express realtime	Live connections and event delivery	Touch the database, or decide anything
Redis	Carrying events from Laravel to Express; queue backing	Store anything that matters
PostgreSQL	Every fact the product asserts	—
React	Normal data from Laravel, live updates from Express	Own business rules

The load-bearing constraint: **the realtime service has no database credentials at all**. It cannot read an incident and it cannot decide who may see one — it verifies a short-lived signed ticket and forwards whatever Redis hands it. That single constraint keeps the split honest.

4. Prerequisites

Tool	Version	Why
PHP + Composer	PHP ^8.3, Composer 2.x	The Laravel API (required only if you run it without Docker)
Node.js + npm	≥ 22	Realtime service, web SPA, e2e tests
Docker Desktop (or any Docker engine)	with Compose v2	Running the whole stack (recommended path)
Make	any	Optional convenience wrappers (GNU make; on Windows: chocolatey make, or just use the commands directly)
Git	any	Version control; CI uses git SHAs as image versions
Redis	7.x (optional)	Only for running "live updates without Docker" — see §13

Note on Windows: the repo is developed on both Linux and Windows. Redis has no official Windows build; the maintained port at github.com/tporadowski/redis is a portable ZIP needing no admin rights (used in the "without Docker" path). Laravel Horizon needs `pcntl/posix` — Linux only — so on Windows use `php artisan queue:work` instead.

5. Scaffolding the repository

Create the folder and the top-level layout. The final structure is:

```

IncidentFlow/
├─ api/           Laravel 12 – REST API, domain logic, queue workers
├─ realtime/      Node 22 + Express – SSE and WebSocket fan-out
├─ web/           React 19 + TypeScript + Vite – the SPA
├─ infra/nginx/   Reverse proxy: one origin for everything
├─ infra/keys/    Empty placeholder; gitignored – real keys live in a Docker volume
├─ scripts/       Reserved, currently empty
├─ docs/          Architecture, API reference, runbook, ADRs, OpenAPI spec
├─ .github/workflows/ GitHub Actions CI
├─ Makefile       Common task wrappers
├─ docker-compose.yml   Dev stack
├─ docker-compose.prod.yml Production overlay
├─ .editorconfig  Shared formatting rules
├─ .gitignore     Root exclusions
└─ README.md

```

```
$ mkdir IncidentFlow && cd IncidentFlow
```

```
$ git init
```

Then create the three sub-projects (details in their own sections). The root files:

`.editorconfig`

```

root = true
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 4
insert_final_newline = true
trim_trailing_whitespace = true

[*.{js,jsx,ts,tsx,json,css,html,yml,yaml}] indent_size = 2
[*.md] trim_trailing_whitespace = false
[*.{sh,bash}] indent_size = 2
[Makefile] indent_style = tab
[*.php] indent_size = 4

```

`.gitignore` (key lines)

```

# Secrets
.env, .env.*      (keep !.env.example and !.env.local.example)
*.pem, *.key      (keep !**/*.example.pem)
api/storage/keys/
infra/keys/

# Dependencies / build output
node_modules/ vendor/ dist/ build/ .vite/
*.sqlite *.sqlite-journal *.sqlite3
coverage/ web/playwright-report/ web/test-results/
api/.phpunit.result.cache api/.phpunit.cache/
api/storage/app/** api/storage/framework/{cache,sessions,views}/** api/storage/logs/**
api/bootstrap/cache/*.php api/public/hot api/public/storage

```

Only `api/.gitignore` exists per-project (the standard Laravel one); `web/` and `realtime/` rely on the root file.

6. Part 1 — Laravel API (backend)

6.1 Creating the project & dependencies

```
$ composer create-project laravel/laravel api "^12.0"
```

Then require the runtime and development packages (final `composer.json` reflects these):

```
$ cd api
```

```
$ composer require laravel/horizon:"^5.30" predis/predis:"^2.2" firebase/php-jwt:"^6.11"
```

```
$ composer require --dev fakerphp/faker:"^1.23" laravel/pail:"^1.2.2" laravel/pint:"^1.18" mockery/mockery:"^1.6"  
nunomaduro/collision:"^8.6" phpunit/phpunit:"^11.5.3"
```

The final `composer.json` scripts and config:

```
"require": {  
    "php": "^8.3",  
    "ext-json": "*", "ext-mbstring": "*", "ext-openssl": "*", "ext-pdo": "*",  
    "firebase/php-jwt": "^6.11",  
    "laravel/framework": "^12.0",  
    "laravel/horizon": "^5.30",  
    "laravel/tinker": "^2.10.1",  
    "predis/predis": "^2.2"  
},  
"require-dev": { "fakerphp/faker": "^1.23", "laravel/pail": "^1.2.2",  
    "laravel/pint": "^1.18", "mockery/mockery": "^1.6",  
    "nunomaduro/collision": "^8.6", "phpunit/phpunit": "^11.5.3" },  
"scripts": {  
    "post-autoload-dump": ["Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",  
        "@php artisan package:discover --ansi"],  
    "lint": "pint --test", "fix": "pint", "test": "phpunit",  
    "check": ["@lint", "@test"]  
},  
"config": {  
    "optimize-autoloader": true, "preferred-install": "dist", "sort-packages": true,  
    "allow-plugins": { "php-http/discovery": true },  
    "platform": { "php": "8.3.0", "ext-pcntl": "8.3.0", "ext-posix": "8.3.0" }  
}
```

The `platform` pins make Composer resolve dependencies as if `pcntl/posix` exist even on Windows (they are needed by Horizon at runtime on Linux; tests never call them).

Run these once for a working local base:

```
$ php artisan key:generate
```

```
$ php artisan migrate
```

```
$ php artisan jwt:keys
```

```
$ php artisan vendor:publish --tag=horizon-assets
```

```
$ php artisan vendor:publish --tag=horizon-config
```

JWT keys: `jwt:keys` is a custom command (see §6.14) that generates the RS256 pair into `storage/keys/`. Horizon is registered in `bootstrap/providers.php` (or `config/app.php`).

6.2 Environment files

Three environment files exist. `.env` (dev, gitignored) drives `php artisan serve`. The Docker path injects everything through Compose instead.

`.env.example` — the canonical documented contract (abridged)

```
APP_NAME=IncidentFlow
APP_ENV=local
APP_KEY=
APP_URL=http://localhost:8000
APP_FRONTEND_URL=http://localhost:5173
APP_VERSION=dev

LOG_CHANNEL=stack
LOG_STACK=json
LOG_LEVEL=debug

DB_CONNECTION=sqlite           # default: SQLite for local dev/tests
# DB_CONNECTION=pgsql         # or PostgreSQL for parity with CI

CACHE_STORE=database           # redis when live updates are enabled
QUEUE_CONNECTION=database      # redis when live updates are enabled
SESSION_DRIVER=database

REDIS_CLIENT=predis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379

REALTIME_ENABLED=false         # true when Redis is running
REALTIME_CHANNEL_PREFIX=incidentflow

JWT_PRIVATE_KEY_PATH=storage/keys/jwt-private.pem
JWT_PUBLIC_KEY_PATH=storage/keys/jwt-public.pem
JWT_PASSPHRASE=
JWT_REFRESH_COOKIE_SECURE=false
JWT_REFRESH_COOKIE_SAMESITE=lax

MAIL_MAILER=log
```

`.env.local.example` — the "with live updates" variant

Identical, except `DB_CONNECTION=sqlite` stays, and the Redis block at the bottom is uncommented:

```
# --- Redis block: uncomment for live updates without Docker ---
REDIS_CLIENT=predis
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis
REALTIME_ENABLED=true
REALTIME_CHANNEL_PREFIX=incidentflow
```

6.3 Configuration (config/)

Beyond the framework defaults, these files were created or customized:

`config/jwt.php` — the token contract

```
return [
    'keys' => [
        'private' => env('JWT_PRIVATE_KEY'),           // inline PEM / base64
        'public'  => env('JWT_PUBLIC_KEY'),           // overrides paths
        'private_path' => env('JWT_PRIVATE_KEY_PATH', storage_path('keys/jwt-private.pem')),
        'public_path' => env('JWT_PUBLIC_KEY_PATH', storage_path('keys/jwt-public.pem')),
        'passphrase' => env('JWT_PASSPHRASE'),
    ],
    'algorithm' => 'RS256',                          // asymmetric: realtime gets only the public half
    'issuer' => 'incidentflow-api',
    'audiences' => ['api' => 'incidentflow-api', 'realtime' => 'incidentflow-realtime'],
    'ttl' => ['access' => 900, 'refresh' => 2_592_000, 'realtime' => 60],
    'leeway' => 30,                                    // seconds of clock skew tolerated
    'denylist' => ['enabled' => true, 'cache_store' => null,
        'prefix' => 'jwt:denylist:'],
    'refresh_cookie' => ['name' => 'incidentflow_refresh', 'path' => '/api/v1/auth',
        'secure' => true, 'same_site' => 'strict'],
];
```

`config/incidents.php` — domain knobs

```
'pagination' => ['default' => 25, 'max' => 100],
'export' => ['max_rows' => env('INCIDENT_EXPORT_MAX_ROWS', 50_000),
    'chunk_size' => 500],
'notifications' => ['stale_after_minutes' => 5, 'max_attempts' => 5],
'idempotency' => ['retention_hours' => 24],
'retention' => ['read_notification_days' => 90,
    'audit_log_days' => null], // null = keep forever, the default and the point
```

`config/realtime.php`

```
return [
    'enabled' => env('REALTIME_ENABLED', false),
    'connection' => 'realtime', // a named Redis connection in config/database.php
    'channel_prefix' => env('REALTIME_CHANNEL_PREFIX', 'incidentflow'),
    'public_url' => '/realtime',
];
```

Also touched: `config/auth.php` (the api guard uses driver jwt, see §6.10), `config/database.php` (adds the realtime Redis connection), `config/queue.php` (retry_after 90), `config/logging.php` (stack/json channels), `config/app.php` (version from APP_VERSION, frontend_url).

6.4 bootstrap/app.php — the wiring

Everything global is declared here (Laravel 12 style). Key parts:

```

return Application::configure(basePath: dirname(__DIR__))
    →withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
        apiPrefix: 'api/v1', // versioned from day one
    )
    →withMiddleware(function (Middleware $middleware): void {
        $middleware→trustProxies(at: '*'); // safe: nginx overwrites X-Forwarded-For
        $middleware→api(prepend: [
            ForceJsonResponse::class,
            AssignRequestId::class,
            NormalizeBooleanParameters::class, // must run before validation
        ]);
        $middleware→alias([
            'auth.jwt' ⇒ AuthenticateJwt::class,
            'organization' ⇒ EnsureOrganizationContext::class,
            'idempotency' ⇒ HandleIdempotency::class,
        ]);
        $middleware→validateCsrfTokens(except: ['api/*']); // stateless API; Horizon keeps CSRF
    })
    →withExceptions(function (Exceptions $exceptions): void {
        $exceptions→render(using: ApiExceptionHandler::class);
        $exceptions→context(fn () ⇒ ['request_id' ⇒ app(RequestContext::class)→requestId()]);
        $exceptions→dontReport([DomainException::class, InvalidTokenException::class]);
    })→create();

```

`trustProxies(at: '*')` is safe only because nothing else can reach the container — nginx overwrites (never appends) `X-Forwarded-For`. Without it, rate limiting would bucket everyone under the proxy IP.

6.5 HTTP middleware (6 classes) — [app/Http/Middleware/](#)

Class	Job	Failure behaviour
<code>AssignRequestId</code>	Reads/validates <code>X-Request-Id</code> (max 64 chars) or mints one; shares it into log context; echoes it back on the response	— (generates)
<code>ForceJsonResponse</code>	Forces <code>Accept: application/json</code> so every error renders as the JSON envelope	—
<code>NormalizeBooleanParameters</code>	Converts string booleans for an explicit allowlist (<code>active_only</code> , <code>assignable_only</code> , <code>public</code> , <code>public_only</code> , <code>unread_only</code> , <code>include_deleted</code> , <code>is_active</code> , <code>is_public</code>) — truthy <code>[true, yes, on]</code> , falsy <code>[false, no, off, '']</code> . No blanket rewrite.	—

AuthenticateJwt	Resolves the Bearer token via the <code>jwt</code> guard; distinguishes coarse failure reasons only	401 JSON: <code>token_expired</code> / <code>token_revoked</code> / <code>unauthenticated</code> (never signature detail — attacker-probing defence)
EnsureOrganizationContext	Reads <code>X-Organization</code> header, verifies membership, binds the <code>Organization</code> into the container. Null users (register/login) pass through.	Missing header → 400 <code>organization_required</code> ; not a member → 404 not 403 (leaks nothing about other tenants)
HandleIdempotency	Idempotency-key enforcement (see §6.9 / note below)	400 <code>idempotency.key_required</code> , 409 <code>idempotency.in_progress</code> , 422 <code>idempotency.payload_mismatch</code>

How HandleIdempotency works (196 lines): the client sends `Idempotency-Key` (mode `required` on incident create, `optional` elsewhere). The middleware *pre-inserts* an `in_progress` row scoped to (`user_id`, `endpoint`, `key`) — the unique index is the concurrency control, and a second concurrent writer loses the INSERT and gets 409. A body hash is stored; a retry with a different body → 422. On handler success the response is stored and replayed **byte-identically** with `Idempotent-Replayed: true`; on handler failure the key is deleted so a genuine retry after a 500 succeeds. Keys expire after 24 h (pruned nightly).

6.6 Enums — the domain vocabulary ([app/Enums/](#))

Nine PHP enums. The two load-bearing ones:

Enum	Values	Behaviour
IncidentStatus	Open, Acknowledged, Mitigated, Resolved, Closed	<code>allowedTransitions()</code> : Open→[Acknowledged, Resolved]; Acknowledged→[Mitigated, Resolved]; Mitigated→[Resolved, Acknowledged]; Resolved→[Closed, reopen]; Closed→[] (terminal). Also <code>isActive()</code> , <code>isReopeningFrom()</code> .
IncidentSeverity	Sev1 .. Sev4	<code>label()</code> "SEV-1 — Critical"...; <code>weight()</code> 1–4; <code>acknowledgementTargetMinutes()</code> (SEV-1 = 5, SEV-2 = 15); <code>requiresImmediateNotification()</code> , <code>requiresPostmortem()</code> (sev1/2).
IncidentEventType	18 cases: Created, StatusChanged, SeverityChanged, Updated, Assigned, Unassigned, CommanderChanged, Commented, UpdatePosted, Acknowledged, Mitigated, Resolved, Closed, Reopened, PostmortemDrafted, PostmortemPublished, Deleted, Restored	<code>describe()</code> → human prose for email bodies; <code>forStatus()</code> → canonical event per transition; <code>isNotifiable()</code> gates notification rows.
Permission	22 values: <code>incident.view</code> , <code>incident.create</code> , <code>incident.update</code> , <code>incident.transition</code> ,	The atomic unit of authorization.

	incident.assign, incident.command, incident.delete, comment.create, comment.moderate, update.create, postmortem.view, postmortem.manage, postmortem.publish, service.view, service.manage, member.view, member.manage, metrics.view, audit.view, export.run, organization.manage	
OrganizationRole	Viewer, Reporter, Responder, IncidentCommander, Administrator	grants(Permission) — a cumulative superset ladder (Viewer = read-only; Reporter adds create/comment; Administrator grants all). canBeAssigned() = Responder and above.
AssigneeRole	Responder, Scribe, Communications, SubjectMatterExpert	Roster roles on an incident.
NotificationChannel	Email, InApp	isQueued() = Email only.
NotificationStatus	Pending, Queued, Sent, Failed, Read	Terminal: Sent/Failed/Read.
PostmortemStatus	Draft, InReview, Published	Published is terminal; isEditable().

Deliberately **no database enums**: severities/statuses are short strings plus PHP enums. Adding a severity then never requires an ALTER TYPE (which takes an exclusive lock on a hot table).

6.7 Migrations — the schema

Eight migration files under `database/migrations/`. Framework ones first (0001_01_01_000000_create_users_table, ...cache_table, ...jobs_table), then four dated domain files. Create them with:

```
$ php artisan make:migration create_organizations_table
$ php artisan make:migration create_services_table
$ php artisan make:migration create_incidents_table
$ php artisan make:migration create_incident_activity_tables
$ php artisan make:migration create_platform_tables
```

2026_01_01_000100_create_organizations_table

```
organizations: id, name, slug (unique), timezone(64) default 'UTC',
  incident_sequence unsignedBigInteger default 0,      // per-tenant counter, row-lock allocated
  settings json nullable, timestampsTz, softDeletesTz

organization_members: organization_id FK cascade, user_id FK cascade,
  role string(32) (NOT an enum - role change = code deploy, never ALTER TYPE),
  invited_by FK nullable nullOnDelete, joined_at timestampTz nullable,
  unique [organization_id, user_id], index [user_id], index [organization_id, role]
```

2026_01_01_000200_create_services_table

```
services: organization_id FK cascade, name, slug(128), description text nullable,
  owner_team nullable, tier unsignedTinyInteger default 3,
  is_active bool default true, timestampsTz, softDeletesTz,
  unique [organization_id, slug], index [organization_id, is_active]
```

2026_01_01_000300_create_incidents_table

```
incidents: organization_id FK cascade, service_id FK nullable nullOnDelete,
  reference string(32), title string(200), description text nullable, impact text nullable,
  severity string(8), status string(16),
  reported_by FK nullable nullOnDelete, commander_id FK nullable nullOnDelete,
  detected_at / acknowledged_at / mitigated_at / resolved_at / closed_at timestampTz nullable,
  time_to_acknowledge_seconds / time_to_resolve_seconds unsignedInteger nullable,
  source string(32) default 'web', external_reference string nullable,
  timestampsTz, softDeletesTz,
  unique [organization_id, reference],
  index [organization_id, status, created_at],      // covers the list view's default query
  index [organization_id, severity, created_at],
  index [organization_id, created_at], index [service_id, status], index [commander_id]

incident_assignees: incident_id FK cascade, user_id FK cascade,
  role string(32) default 'responder', assigned_by FK nullable nullOnDelete,
  assigned_at timestampTz, timestampsTz,
  unique [incident_id, user_id], index [user_id, incident_id]
  // holds the CURRENT roster only; history lives in incident_events
```

2026_01_01_000400_create_incident_activity_tables

```
incident_updates: incident_id FK cascade, user_id FK nullable nullOnDelete,
previous_status string(16) nullable, status string(16) nullable,
message text, is_public bool default false, timestampsTz,
index [incident_id, created_at]

incident_comments: incident_id FK cascade, user_id FK nullable nullOnDelete,
body text, edited_at timestampTz nullable, timestampsTz, softDeletesTz,
index [incident_id, created_at]

incident_events: ← APPEND-ONLY
id, ulid unique, incident_id FK cascade, organization_id FK cascade
(denormalized so org feeds and the realtime publisher never join back),
type string(64), actor_id FK nullable nullOnDelete, actor_name nullable (snapshot),
payload json nullable, request_id string(64) nullable, occurred_at timestampTz,
created_at timestampTz nullable
← NO updated_at, NO deleted_at
index [incident_id, id], index [organization_id, occurred_at], index [organization_id, type]

postmortems: incident_id FK UNIQUE cascade, organization_id FK cascade,
author_id FK nullable nullOnDelete, title, summary/root_cause/contributing_factors/
impact/resolution/detection_notes/lessons_learned text nullable,
action_items json nullable, status string(16) default 'draft',
published_at timestampTz nullable, timestampsTz, softDeletesTz,
index [organization_id, status]
```

2026_01_01_000500_create_platform_tables

```
notifications: id, ulid unique, organization_id FK cascade, user_id FK cascade,
  incident_id FK nullable cascade, channel string(16), type string(64),
  subject string, body text, payload json nullable,
  status string(16) default 'pending', attempts unsignedTinyInteger default 0,
  last_error text nullable, sent_at / read_at timestampTz nullable, timestampsTz,
  index [user_id, read_at, created_at], index [status, channel], index [organization_id, created_at]

audit_logs: ← APPEND-ONLY
  id, ulid unique, organization_id FK nullable nullOnDelete,
  actor_id FK nullable nullOnDelete, actor_email nullable (snapshot),
  action string(64), auditable_type string(64) nullable, auditable_id unsignedBigInteger nullable,
  changes json nullable ({"before":..., "after":...}, sensitive keys stripped),
  ip_address string(45) nullable, user_agent string(512) nullable,
  request_id string(64) nullable, created_at timestampTz nullable
  index [organization_id, created_at], index [auditable_type, auditable_id],
  index [actor_id, created_at], index [action]

idempotency_keys: organization_id FK nullable cascade, user_id FK nullable cascade,
  key string(255), endpoint string(160), request_hash string(64),
  status string(16) default 'in_progress',
  response_status unsignedSmallInteger nullable, response_body json nullable,
  resource_type string(64) nullable, resource_id unsignedBigInteger nullable,
  expires_at timestampTz, timestampsTz,
  unique [user_id, endpoint, key], ← the load-bearing concurrency control
  index [expires_at]

refresh_tokens: user_id FK cascade, token_hash string(64) UNIQUE (SHA-256, never raw),
  family_id uuid, parent_id FK nullable self-referential nullOnDelete,
  expires_at, revoked_at nullable, revoked_reason string(64) nullable,
  user_agent string(512) nullable, ip_address string(45) nullable, timestampsTz,
  index [user_id, revoked_at], index [family_id], index [expires_at]
```

Append-only by construction (three layers): (1) schema — no updated_at/deleted_at on incident_events and audit_logs; (2) models — updating/deleting throw; (3) API — no mutating route. Corrections are appended as new events, never applied in place.

6.8 Models ([app/Models/](#))

Created with `php artisan make:model Name -mfs` (model + migration + factory + seeder, where wanted).
15 models plus one trait:

Model	Highlights
Concerns/BelongsToOrganization (trait)	organization() + scopeForOrganization(). Explicit tenancy — deliberately no global scope (ADR-0002).
User	memberships() belongsToMany Organization via pivot role; roleIn(Organization); is_active checked by the JWT guard on every request.
Organization	allocateIncidentNumber() uses lockForUpdate() on its own row inside a transaction (no global sequence leak, O(1)).

OrganizationMember	The (user, org) → role edge — every authorization decision resolves to a row in this table.
Service	booted() hook auto-slugs (uniqueSlugFor, suffixed -2/-3); scopeActive; tiers.
Incident (238 lines)	Route key id; referenceFor() = 'INC-' ' .str_pad(\$sequence, 4, '0', STR_PAD_LEFT); relations service/reporter/commander/assignees/updates/comments/events/postmortem; requiresPostmortem(); scopes whereStatusIn, whereSeverityIn (no-op on empty), assignedTo() (assignment or commander), search() (escapes %/_ , whereLike caseSensitive:false → ILIKE on PG / LIKE on SQLite); SORTABLE const; SoftDeletes.
IncidentEvent	HasUlid; UPDATED_AT = null; throws on update/delete (see §6.7 note).
IncidentUpdate	The communications log (optionally public) — distinct from comments.
IncidentComment	Soft-deletable; edited_at.
IncidentAssignee	Pivot model (role, assigned_by, assigned_at).
Postmortem	action_items json array; status enum; one per incident.
Notification	Route key ulid; markRead(); scopeUnread. Row-first retries: a failed email retries the <i>row</i> , never the incident update.
AuditLog	Route key ulid; UPDATED_AT = null; append-only.
RefreshToken	hash() = SHA-256; revoke(\$reason); scopeFamily.
IdempotencyKey	Status constants; hashPayload(); scopeExpired.

6.9 Services — the business logic ([app/Services/](#))

Incidents/IncidentService (527 lines) — the single write path

Every incident mutation runs through here with one pattern: **(1) open transaction → (2) re-read the row FOR UPDATE (stale-copy guard) → (3) mutate + exactly one incident_events row + one audit_logs row + notification rows, atomically → (4) commit → (5) only then publish to Redis and enqueue email.**

Side effects are collected in a PendingEffects object (two arrays: timelineEvents, notifications) and flushed after DB::transaction() returns. No framework magic, no afterCommit hook whose behaviour differs under test.

```
$this->withEffects(function () use (...) {
    $incident = $this->lock($incident);           // lockForUpdate()→firstOrFail()
    ... validate transition against the state machine ...
    $incident->update([ ... ]);                   // status + acknowledged_at/resolved_at +
    durations
    $this->timeline->record($incident, $eventType, $actor, [ ... ]);
    $this->audit->record('incident.status_changed', $incident, ...);
    $effects->notifications = $this->dispatcher->prepare($incident, $eventType, ...);
});                                               // commit happens inside withEffects()
// after commit:
// foreach ($effects->timelineEvents) RealtimePublisher->publish(...)
// $this->dispatcher->dispatch($effects->notifications)
```

Key rules encoded in `transition()`: same status → `incident.status_unchanged`; terminal → `incident.terminal_status`; illegal move → `incident.illegal_transition` with `{from, to, allowed}`. `acknowledged_at` is never overwritten (TTA = first human response); `resolved_at` is cleared on reopen; resolving without an explicit acknowledgement back-fills TTA = TTR so incidents never silently drop out of the MTTA average. A non-empty transition note also creates an `IncidentUpdate` row. `setCommander()` auto-assigns the commander as a Responder. Comments/updates record events with 200-char excerpts (`mb_strimwidth(..., 0, 200, '...')`).

Incidents/TimelineRecorder

The single writer of `incident_events`: snapshots `actor_name` (timeline must read correctly after a user is deleted) and stamps `request_id` from `RequestContext`.

Realtime/RealtimeEvent + RealtimePublisher

The envelope (version 1) that crosses the wire:

```
{ version, id (ulid – doubles as SSE event id), type ("incident.status_changed"),
  organizationId, incidentId, occurredAt, actor {id, name}, requestId, payload }
```

`RealtimePublisher` publishes to the Redis connection named `realtime` on channel `{prefix}:org:{organization_id}`. Failures are **logged, never thrown** — the API stays up without Redis (the product degrades to refetching). Called only after commit.

Notifications/NotificationDispatcher (223 lines)

`prepare()` (inside the transaction): creates notification rows; skips non-notifiable event types; rejects the actor as a recipient ("Nobody needs an email telling them what they just did"); in-app rows are written already **Sent**; email rows **Pending**. `dispatch()` (after commit): enqueues `SendIncidentNotification` per queued row. Recipients: assignees (for **Assigned**); otherwise commander + reporter + assignees; **Created** on high severity or escalation into sev1/sev2 also pages every assignable active member. `stalePending()` re-queues pending rows older than 5 minutes (swept by the scheduler).

Metrics/MetricsService

MTTA/MTTR come from the **denormalized columns** (`time_to_acknowledge_seconds`, `time_to_resolve_seconds`), so aggregates are a plain SQL `AVG()` and stay stable even if an admin later corrects a timestamp. Percentiles (p50/p90/p95) and daily buckets are computed in PHP over a capped row set (`MAX_SAMPLE_ROWS = 20_000`) for SQLite/PostgreSQL parity (`percentile_cont/date_trunc` don't exist in SQLite). When a tenant outgrows the cap, the replacement is a nightly rollup table — and the output shape of `MetricsService` is already exactly what such a table would store.

Audit/AuditLogger

`record(action, auditable, actor, organizationId, changes)` inside the same transaction; strips REDACTED keys (`password`, `password_confirmation`, `remember_token`, `token`, `token_hash`, `secret`, `api_key`, `private_key`); snapshots `actor_email`; captures IP/user-agent/request-id.

Support/RequestContext

Singleton carrying `request_id` (defaults to a fresh uuid) + client info, shared into log context.

6.10 Authentication (JWT guard, tokens)

Three credentials, three threat models:

Credential	Lifetime	Where it lives	Revocable
Access token (JWT, aud: <code>api</code>)	15 min	Browser memory only	Via cache denylist on logout
Refresh token (opaque)	30 days	HttpOnly cookie	Yes — row in the database
Realtime ticket (JWT, aud: <code>realtime</code>)	60 s	Query string	Expires faster than it matters

RS256, not HS256: the realtime service must verify tokens the API mints. With a shared secret it could also *forge* them. Asymmetric: the realtime tier holds only the public half — it can verify but never sign.

The access token carries no roles. It answers "who is this?"; the database answers "what may they do?" — one indexed lookup per request, and removing a role takes effect on the very next request. The realtime ticket *does* carry role + organization (the Express service has no DB to ask); the cost is bounded staleness of at most 60 s, a documented trade.

`app/Auth/JwtGuard.php` (137 lines)

A real `Illuminate\Contracts\Auth\Guard` registered via `Auth::extend('jwt', ...)` in `AppServiceProvider::boot()` — not middleware calling `auth()→setUser()` — so policies, `$request→user()`, and Gate checks in queued jobs all work. Bearer token → `TokenService::decodeAccessToken` → user must exist and be `is_active` (deactivation is immediate). Exposes `lastFailure()` and `claims()`.

`app/Services/Auth/TokenService.php` (363 lines)

- **Access:** claims `iss`, `aud`, `sub`, `jti`, `iat`, `nbfi`, `exp` (900 s) + convenience `name/email` (never used for authorization).
- **Realtime ticket:** 60 s, aud: `incidentflow-realtime`, claims `org_id`, `role`, `name`.
- **Refresh token:** 256-bit CSPRNG opaque string, stored SHA-256-hashed; `family_id` (uuid) chains rotations; 30-day expiry; user-agent/IP recorded.
- **Rotation with reuse detection:** if a presented token is already revoked, the whole family is revoked in its own transaction (so the revocation can't be rolled back by the throwing rotation), then `InvalidTokenException::reused()` is thrown. Inside the rotation transaction: `lockForUpdate`, expiry → `revoke('expired')`, inactive user → `revoke('subject_inactive')`.
- **Denylist:** `denyAccessToken()` stores the `jti` in cache for exactly `secondsUntilExpiry()`; checked on every decode.
- **Decode:** sets `JWT::$leeway` (30 s); maps firebase exceptions to reasons; **issuer and audience validated explicitly** — "skipping these is how a ticket gets replayed as an access token". `sub` must be digits, `jti` present.

`app/Services/Auth/KeyProvider.php`

Memoized resolution of inline key (raw PEM, PEM with literal `\n`, or base64) vs file path; anchors relative paths to `base_path()` so it works under artisan serve, php-fpm and queue workers; validates the PEM markers so private/public can't be swapped.

`app/Services/Auth/IssuedToken.php` / `TokenClaims.php`

Readonly value objects; `IssuedToken::expiresIn()` lets the frontend know expiry without decoding the JWT.

AuthController

`register` creates **user + first organization + membership in one transaction** (creator becomes Administrator). The refresh cookie: name `incidentflow_refresh`, path `/api/v1/auth`, HttpOnly, Secure (prod), SameSite=strict. `refresh` rotates; `logout` revokes + denylists; `logout-all` revokes every family.

The refresh cookie is scoped to `/api/v1/auth` — never widen that path; it is what keeps the cookie off the hundreds of API calls with no use for it (runbook security note).

6.11 Controllers, Requests, Resources (`app/Http/...`)

19 controllers in `Api/V1`. Route-level authorization comes from the three middleware layers; *object-level* authorization comes from policies invoked per action (see §6.12).

Controller	Key behaviours
HealthController	<code>live</code> touches nothing; <code>ready</code> checks DB (<code>select 1</code>) + cache, reports Redis without failing on it (documented degradation choice).
AuthController	<code>register</code> / <code>login</code> / <code>refresh</code> / <code>me</code> / <code>logout</code> / <code>logout-all</code> (see §6.10).
OrganizationController	<code>index</code> sits <i>outside</i> the organization middleware (a client calls it precisely before it has a tenant). <code>show/update</code> gated (OrganizationManage).
MemberController	Filters <code>role</code> , <code>q</code> , <code>per_page</code> (max 200), <code>assignable_only</code> ; escalation guard on role changes (may not grant above own role, may not change own role).
ServiceController	CRUD + <code>withCount(incidents ⇒ activeOnly)</code> , ordered tier then name.
IncidentController	List with filters (<code>status[]</code> , <code>severity[]</code> , <code>q</code> , <code>from/to</code> , whitelisted sort, stable tie-break by id); <code>store</code> requires idempotency; <code>update</code> = descriptive fields only (<code>severity/status</code> deliberately absent — they route around the state machine); soft delete.
IncidentStatusController	Single <code>__invoke: Gate::authorize('transition')</code> , calls service. Status is a separate endpoint, not a PATCH field — state-machine routing.
IncidentSeverityController	Gated <code>command</code> ; downgrades require a reason <code>min:10</code> ("downgrades must be justified").
IncidentCommanderController	PUT semantics (setting twice = once); <code>commander_id</code> must exist in <code>organization_members</code> (tenant-scoped).
IncidentAssigneeController	Assignment restricted to <code>assignable_only</code> roles; non-eligible → 422 <code>assignee_not_eligible</code> with <code>{required_minimum_role: "responder"}</code> .

IncidentEventController	Timeline listing, read-only; cursor pagination (timeline grows while you read; LIMIT/OFFSET would repeat rows); after_id resync for stream gaps; per_page max 200.
IncidentUpdateController	Communications log list + store (message 3–5000, public flag).
IncidentCommentController	List + store (1–10 000 chars) + soft-delete with audit comment.deleted (before.body).
IncidentExportController (163 lines)	Streams CSV via chunkById (memory stays at a few hundred rows); CSV-injection defence (cells starting = + - @ \t \r get a ' prefix); UTF-8 BOM; max_rows 50_000 , chunk_size 500 ; audit entry incident.exported ; gated export.run .
PostmortemController	List (cursor), show, upsert (PUT — one postmortem per incident), publish gated by content: 422 postmortem.incomplete with missing_sections .
MetricsController	summary + trends , MAX_WINDOW_DAYS = 366 .
NotificationController	"No policy is involved and none should be: authorization here is the query itself" — every read/write scoped to user_id = the caller . Marking someone else's notification read → 404 (caller must not learn the id exists).
AuditLogController	Read-only, administrators only; filters action/actor/auditable/request_id/from/to; cursor-paginated desc.
RealtimeTicketController	POST → 60-second ticket with org + role embedded (see §6.10).

Form Requests (app/Http/Requests/)

- **LoginRequest**: email (trimmed, lowercased, max 255) + password (max 255, **no** Password rules — don't leak the policy, don't reject legacy credentials).
- **RegisterRequest**: name 2–120; email rfc+strict unique; password **confirmed** via **Password::defaults()** (min 12, **uncompromised()** only in production/staging — it is a network call); organization_name 2–120.
- **StoreIncidentRequest**: title 5–200, description ≤ 20 000, impact ≤ 5 000, severity in enum, **service_id tenant-scoped** exists, commander scoped to org members.
- **UpdateIncidentRequest**: no severity/status (see above); **detected_at before_or_equal:now**.
- **TransitionIncidentRequest**: status in enum + note ≤ 5 000 + public bool.
- **IndexIncidentsRequest**: everything filterable, whitelisted sort, per_page ≤ 100. (Booleans arrive normalized by the middleware.)

Resources (app/Http/Resources/) — 11

IncidentResource returns severity/status as *objects* (**{value, label, ...}**) so the UI renders server truth; includes **allowed_transitions** and the five clock timestamps + two durations. **OrganizationResource** casts empty **settings** to (**object**) so it serializes as {}, never [] (a typed client rejects the array). **IncidentCommentResource** includes the policy-resolved **can_delete** flag.

6.12 Policies — authorization (app/Policies/)

Three policy classes + one trait (**Concerns/ChecksOrganizationPermissions**):

- `checksOrganizationPermissions::allows($permission)` — request-org-scoped check; `allowsWithin($permission, $resource)` — **first proves resource org == request org** (the broken-object-level-authorization guard), then checks the permission. Policies check *permissions*, never roles.
- **IncidentPolicy**: `viewAny/view/create/update/transition/assign/command/comment/postUpdate/deleteComment/export` — each delegates to `allowsWithin`.
- **OrganizationPolicy**: `view, update (OrganizationManage), viewMembers/manageMembers, viewAudit, viewMetrics, assignRole` with the escalation guard (cannot grant above own role; cannot change own role). Registered for both **Organization** and **OrganizationMember**.
- **ServicePolicy, PostmortemPolicy** (`view/manage/publish`).

Why no global tenant scope (ADR-0002): a global Eloquent scope makes queries safe "by accident" — it reads "current tenant" from ambient state that only exists inside an HTTP request. In a queue worker, an artisan command or the scheduler it fails open: a report quietly including every customer's incidents. Instead `EnsureOrganizationContext` verifies the tenant once per request and every query names it via `forOrganization()` — the safety is visible at the call site.

6.13 Jobs, Mail and the notification pipeline

`app/Jobs/SendIncidentNotification.php`

Constructor takes `int $notificationId` — never a model — so the row is re-read (with recipient + incident) on every attempt: the job stays small and current. `$tries = 5, $backoff = [10, 30, 120, 300], $timeout = 30`. Marks the row queued → sent/failed with `last_error`. **Delivery only — it never re-applies the incident change** (a retry never re-runs the incident update; it only moves the row to sent).

`app/Mail/IncidentNotificationMail.php`

Message-ID: `{ulid}@domain, References: incident-{id}@domain, tags ['incident-notification']`; links use `config('app.frontend_url')`. The blade template (`resources/views/mail/incident-notification.blade.php` — the only view in the repo) repeats the reference at the bottom in plain text for searchability.

End-to-end flow of one status change (the two load-bearing orderings): status + timeline + audit + notification rows commit *as one transaction*; Redis publish and queue dispatch happen *only after commit* — publishing inside the transaction would announce a state change a rollback then erases.

6.14 Console commands & the scheduler

Three custom commands in `app/Console/Commands/`:

Command	Signature	Behaviour
<code>GenerateJwtKeys</code>	<code>jwt:keys [--force] [--bits=2048] [--path=]</code>	Refuses < 2048 bits; writes 0600-permission files; warns that replacing a pair signs out every session.
<code>PruneExpiredRecords</code>	<code>incidentflow:prune [--dry-run]</code>	Deletes expired idempotency keys, revoked refresh tokens older than 30 days, read notifications older than 90 days.

		Never touches audit_logs or incident_events — permanent by design.
RetryStaleNotifications	notifications:retry-stale {--dry-run}	Re-queues pending rows via <code>NotificationDispatcher::stalePending()</code> — covers process death between <code>prepare()</code> and <code>dispatch()</code> . Safe to run concurrently.

`routes/console.php` — the schedule:

```
Schedule::command('notifications:retry-stale')
    →everyFiveMinutes()→withoutOverlapping()→onOneServer()→runInBackground();
Schedule::command('incidentflow:prune')→dailyAt('03:30')→withoutOverlapping()→onOneServer();
Schedule::command('queue:prune-batches --hours=48')→daily()→onOneServer();
Schedule::command('queue:prune-failed --hours=336')→weekly()→onOneServer();
```

`withoutOverlapping/onOneServer` matter because the scheduler runs on every API container. In Docker it runs as `php artisan schedule:work`.

6.15 Exceptions — one error shape

`app/Exceptions/`:

- `DomainException` — abstract base for client-fault rule violations; carries `errorCode` + `status` + `details`. "The domain says what went wrong, one handler decides how to render it, the frontend switches on `code`."
- `ApiExceptionRenderer` (174 lines) — the single envelope: `{"error": {"code", "message", "details", "request_id"}}`. `ValidationException` → 422 `validation_failed` with fields; `AuthenticationException` → 401; `InvalidTokenException` → 401 `token_{reason}`; `AuthorizationException` → 403 `forbidden`; `ModelNotFound` and 404 → `not_found` (identical — never confirm an id exists in another tenant); 429 → `rate_limited`; default → 500 `internal_error` with generic message in production + `request_id`.
- `InvalidTokenException` — reasons `malformed`, `expired`, `invalid_signature`, `wrong_audience`, `revoked`, `reused`, `unknown_subject`; the reason is logged, never returned in full.
- `IncidentTransitionException` — `illegal(from, to)` → 422 `incident.illegal_transition` with `{from, to, allowed}`; `alreadyInStatus`; `terminal`.
- `IdempotencyConflictException` — `inFlight` → 409 `idempotency.in_progress`; `payloadMismatch` → 422.
- `PostmortemException` — `incomplete($missing)` → 422 with `missing_sections`.

6.16 Routes

`routes/api.php` (137 lines) — mounted under `/api/v1`; three middleware layers stack: `auth.jwt` (who), `organization` (which tenant), `throttle:*` (how often). Authorization is deliberately not middleware. Complete route table:

```

GET /health/live, /health/ready ← unauthenticated probes

POST /auth/register|login|refresh (throttle:auth)
GET /auth/me | POST /auth/logout | POST /auth/logout-all (auth.jwt)

GET /organizations (auth.jwt – outside org middleware)

# everything below: auth.jwt + organization + throttle:api
POST /realtime/ticket (throttle:realtime)
GET|PATCH /organization
GET|POST|PATCH|DELETE /members{/member}
GET|POST|PATCH|DELETE /services{/service}
GET /incidents/export (throttle:exports – DECLARED BEFORE
{incident})
GET|POST /incidents (POST: throttle:writes +
idempotency:required)
GET|PATCH|DELETE /incidents/{incident} (named: incidents.show)
POST /incidents/{incident}/status (throttle:writes + idempotency)
POST /incidents/{incident}/severity
PUT /incidents/{incident}/commander
GET|POST /incidents/{incident}/assignees
DELETE /incidents/{incident}/assignees/{user}
GET /incidents/{incident}/events (cursor-paginated)
GET|POST /incidents/{incident}/updates (POST idempotent)
GET|POST /incidents/{incident}/comments (POST idempotent)
DELETE /comments/{comment}
GET /postmortems | GET|PUT /incidents/{incident}/postmortem | POST .../publish
GET /metrics/summary | /metrics/trends
GET /notifications | POST /notifications/read-all | POST /notifications/{n}/read
GET /audit-logs

```

`/incidents/export` is declared *before* `incidents/{incident}` on purpose: Laravel matches routes in definition order, so a later literal segment would be swallowed by the wildcard and "export" would be looked up as an id.

`routes/web.php` — a single GET `/` returning JSON (`{service, version, documentation, health, message}`) so a misconfigured proxy or wrong-path health check gets a useful answer instead of a Laravel welcome page. Horizon mounts at `/horizon`, gated by `viewHorizon` (defined in `AppServiceProvider`: user is Administrator in at least one org).

Rate limiters (`AppServiceProvider`): `api` 180/min per user or 40/min per IP; `auth` 10/min per IP *and* 5/min per email | ip (bounds credential stuffing without handing out a DoS primitive against named users); `writes` 60/min; `exports` 10/hour; `realtime` 60/min.

6.17 Seeders & factories

`database/seeders/DatabaseSeeder.php` (264 lines):

- Organization "Northwind Logistics" (slug `northwind`); **refuses to run when `APP_ENV=production`**.
- `DEMO_PASSWORD = 'incidentflow'` for six users: `admin`, `commander`, `responder` (×2), `reporter`, `viewer` @`incidentflow.test` (Riya Sharma, Daniel Okoye, Mei Tanaka, Luis Ferreira, Ahmed Hassan, Sofia Rossi).
- Five services (Checkout API tier 1, Shipment Tracking tier 1, Notification Gateway tier 2, Warehouse Sync tier 2, Internal Reporting tier 3).

- **Incident history is generated through the real IncidentService** — a 17-incident script over 90 days exercising the full transaction→timeline→audit→notification path; deliberately uneven ack/resolve minutes so p95 differs visibly from the mean; three incidents still open; resolved ones older than 21 days get Closed; each gets a comment. `Carbon::setTestNow()` is anchored once before the loop (the anchor comment explains how naive time-travel silently spreads 90 days over years).

Factories (8): `UserFactory`, `OrganizationFactory`, `OrganizationMemberFactory`, `ServiceFactory`, `IncidentFactory`, `IncidentEventFactory`, `IncidentUpdateFactory`, `IncidentCommentFactory`.

6.18 Tests (110 tests)

`tests/` — 16 Unit + 94 Feature. Key files: `tests/TestCase.php` generates RSA-2048 test keys into `storage/keys/testing-*.pem` **before** `parent::setUp()` (app not booted, so root derived from `dirname(__DIR__)`); helpers `tenantWithMember()`, `actingAsMember()`, `assertApiError()`.

- `Unit/OrganizationRoleTest` (7) — the permission-ladder contract: every role grants a superset of the one below; Viewer is strictly read-only.
- `Unit/IncidentStatusTest` (9) — the exhaustive transition matrix.
- `Feature/Auth/AuthenticationTest` (16) — register/login/refresh/logout flows, rotation, reuse detection, breached-password rejection.
- Feature tests for incidents (create/update/transition/severity/assignees/commander), timeline, comments, updates, postmortems, metrics, notifications, exports (CSV injection), audit logs, services, members, idempotency, tenancy isolation (cross-tenant probes), rate limiting.

`phpunit.xml` : SQLite in-memory database per test (`DB_CONNECTION=sqlite`, `DB_DATABASE=:memory:`), JSON log channel, bcrypt hasher cost lowered for speed.

The API suite runs on in-memory SQLite locally and against real PostgreSQL 16 in CI — portable-looking schema-builder calls can behave differently on the engine that ships.

6.19 The API Docker image

`api/Dockerfile` — three stages (`# syntax=docker/dockerfile:1.7`):

```

FROM php:8.3-fpm-alpine AS extensions
  apk add --no-cache --virtual .build-deps $PHPIZE_DEPS postgresql-dev linux-headers
  docker-php-ext-install -j"${nproc}" pdo_pgsql pcntl bcmath opcache

FROM composer:2.8 AS vendor
  COPY composer.json composer.lock ./
  RUN --mount=type=cache,target=/tmp/composer-cache \
    composer install --no-dev --no-scripts --no-interaction --prefer-dist --no-progress
  COPY . .
  RUN composer dump-autoload --optimize --classmap-authoritative --no-dev

FROM php:8.3-fpm-alpine AS runtime
  apk add --no-cache postgresql-libs tini
  COPY --from=extensions extensions + conf.d
  COPY --from=vendor /app → /var/www/html (--chown=www-data:www-data)
  COPY docker/php.ini → conf.d/99-incidentflow.ini
  COPY docker/php-fpm.conf → php-fpm.d/zz-incidentflow.conf
  COPY docker/entrypoint.sh → /usr/local/bin/entrypoint
  mkdir storage/framework/{cache,sessions,views} storage/logs storage/keys bootstrap/cache
  ARG APP_VERSION=dev / ENV APP_VERSION=$APP_VERSION
  USER www-data ; EXPOSE 9000
  HEALTHCHECK: php -r "exit(@fsockopen('127.0.0.1', 9000) ? 0 : 1);" (dependency-free)
  ENTRYPOINT ["/sbin/tini", "--", "/usr/local/bin/entrypoint"]
  CMD ["php-fpm"]

```

`docker/entrypoint.sh` (50 lines) — **never runs migrations** (replicas would race; that is the one-shot migrate service's job). It fails fast without `APP_KEY`, waits for PostgreSQL (30 attempts × 2 s), generates JWT keys if the volume is empty, caches config/routes/events when `APP_DEBUG` $\neq$ `true`, then exec `"$@"`.

`docker/php.ini` — memory 256M, errors to stderr, OPcache with `validate_timestamps=0` (immutable image), `expose_php=0ff`, secure session cookie flags.

`docker/php-fpm.conf` — `pm = dynamic` (max 20 children, 500 requests per worker), access log to stderr, `pm.status_path/ping.path`.

7. Part 2 — Realtime service (Node 22 + Express)

The entire reason this service exists: PHP-FPM allocates one whole worker process per in-flight request; ten thousand open SSE streams would mean ten thousand blocked PHP workers — a configuration that cannot exist on any reasonable amount of RAM. Node's event loop holds idle sockets almost for free, so the fan-out tier lives here (ADR-0001). It holds **no database credentials** and **decides nothing**: it verifies a signed 60-second ticket and forwards whatever Redis hands it.

7.1 Project setup

```
$ mkdir realtime && cd realtime
```

```
$ npm init -y
```

```
$ npm install express:"^5.1.0" ioredis:"^5.8.2" jose:"^6.1.0" pino:"^9.13.1" prom-client:"^15.1.3" ws:"^8.18.3"
  zod:"^4.1.14"
```

```
$ npm install -D @types/express:"^5.0.5" @types/node:"^24.10.1" @types/supertest:"^6.0.3" @types/ws:"^8.18.1"
  eslint:"^9.39.1" ioredis-mock:"^8.12.0" supertest:"^7.1.4" tsx:"^4.20.7" typescript:"^5.9.3" typescript-
```

```
eslint:"^8.47.0" vitest:"^3.2.4"
```

package.json — "type": "module", "engines": {"node": "≥ 22"}, scripts:

```
"dev": "tsx watch src/index.ts",
"build": "tsc -p tsconfig.build.json",
"start": "node dist/index.js",
"typecheck": "tsc --noEmit",
"lint": "eslint .",
"test": "vitest run",
"test:watch": "vitest"
```

tsconfig.json: target/lib ES2023, module/moduleResolution NodeNext, strict, noUncheckedIndexedAccess, noImplicitOverride, noFallthroughCasesInSwitch, verbatimModuleSyntax, sourceMap; includes src/**/*.ts and test/**/*.ts. **tsconfig.build.json** extends it with rootDir: src, excludes tests.

eslint.config.js: flat config via tseslint.config(...); no-unused-vars (^_ escape hatch), no-explicit-any error, no-console error — "a stray console.log breaks the JSON log stream".

7.2 **src/config.ts** — the env contract (zod)

Fail-fast: the process refuses to boot on invalid env. Everything is a zod-validated constant:

```
HOST (0.0.0.0), PORT (default 3001), NODE_ENV, LOG_LEVEL,
SERVICE_NAME (incidentflow-realtime),
REDIS_URL (redis://127.0.0.1:6379), REDIS_CHANNEL_PREFIX (incidentflow),
JWT_PUBLIC_KEY | JWT_PUBLIC_KEY_PATH (normalisePem handles raw / \n-escaped / base64),
JWT_ALGORITHM (RS256), JWT_ISSUER (incidentflow-api), JWT_AUDIENCE (incidentflow-realtime),
JWT_CLOCK_TOLERANCE_SECONDS (30),
CORS_ORIGINS (http://localhost:5173),
SSE_HEARTBEAT_MS (15_000), SSE_RETRY_MS (3_000), REPLAY_BUFFER_SIZE (200),
MAX_CONNECTIONS (10_000), MAX_CONNECTIONS_PER_USER (10),
WS_MAX_MESSAGES_PER_MINUTE (120), SHUTDOWN_TIMEOUT_MS (10_000), TRUST_PROXY
```

7.3 **src/redis.ts** — RedisChannelSubscriber (153 lines)

- Dedicated ioredis client in subscribe mode: lazyConnect, maxRetriesPerRequest: null, enableReadyCheck, exponential retry backoff (200 ms base → 10 s cap), reconnectOnError → true.
- On re-connect, restoreChannels() re-subscribes the hub's active channels (idempotent, logged redis.channels_restored).
- handleMessage: JSON.parse → validate with RealtimeEventSchema → drop if version > 1 (unsupported_version — a newer API must not take the fan-out tier down) → forward to onEvent. Outcome counted on the metric.

Redis pub/sub is fire-and-forget; PostgreSQL is the source of truth, so a dropped frame costs a refetch, never data. The loss is made *visible* (replay buffer + stream.gap) rather than hidden. Choosing a lossy transport is fine — hiding the loss is not.

7.4 `src/hub.ts` — the channel hub (337 lines)

- Data structures: `clients` map (`id` → {client, topics}), `byOrganization` / `byUser` sets (dropped when empty), `channelRefCount`s, per-org `replayBuffers` (ring buffer, oldest first).
- **Channel naming:** `{prefix}:org:{orgId}`. A node subscribes to an org's Redis channel only while it has a connected member — released when the last one disconnects (**reference counting**). The alternative, `PSUBSCRIBE incidentflow:org:*`, makes every node receive every tenant's traffic; reference counting is what makes horizontal scaling actually divide load.
- **Limits:** global `maxConnections`, then per-user `maxConnectionsPerUser`; failures → `ConnectionLimitError('global' | 'per_user')` (SSE: 503 + `Retry-After: 5`; WS: close code 1013).
- **Registration:** every client is implicitly subscribed to its own org topic; then `acquireChannel(orgId)` issues a real Redis subscribe only on 0→1. `updateSubscriptions` validates topics against `^(org|incident):[1-9][0-9]{0,17}$` (rejects `org:0`, negatives, `*`, injections) and refuses unsubscribing from the org topic (structural).
- **Dispatch:** remembers the event in the ring buffer, looks up candidates by org, then a defence-in-depth double check — principal's org must equal the event's org (mis-routed publishes must never cross tenant boundaries) and topics must intersect. A delivery throw → `delivery_failure`, connection closed.
- **Replay:** searches the org buffer for the event matching `lastEventId`; returns events after it (topic-filtered) or `null` when the cursor aged out — the caller then sends `stream.gap` so the client refetches from PostgreSQL instead of silently receiving partial history.
- **Shutdown:** `broadcastControl('stream.reconnect', ...)` tells clients to move; `closeAll(reason)`.

7.5 Transports — `src/transport/sse.ts` and `ws.ts`

SSE (the primary browser transport)

- Headers: `text/event-stream; charset=utf-8, Cache-Control: no-cache, no-store, no-transform, Connection: keep-alive, X-Accel-Buffering: no` (stops nginx buffering the stream into silence).
- First frames: `retry: <retryMs>`, then control event `stream.open` with `{connection_id, organization_id, heartbeat_ms}`.
- Heartbeat: comment frame : `heartbeat <ts>\n\n` every 15 s (valid SSE, ignored by EventSource, keeps intermediaries' idle reapers happy); timer unref'd.
- Frame formats — event: `id: <event.id>\nevent: <event.type>\ndata: <JSON>\n\n`; control: `event: <type>\ndata: {"type":...}\n\n`.
- **Backpressure:** if `response.write()` returns false and `writableLength` exceeds 1 MiB (`MAX_BUFFERED_BYTES`), throw — a slow consumer is dropped rather than allowed to grow a shared node's heap.
- Credential and resume cursor: `Authorization` header / `?ticket=`; `Last-Event-ID` header / `?last_event_id=`.

WebSocket (for non-browser consumers)

- **Auth happens during the HTTP upgrade, before the socket exists** — unauthenticated peers get 401 written directly to the raw socket.
- `WebSocketServer({ noServer: true, maxPayload: 16 KiB })` attached to the HTTP server's upgrade event; path must equal `/ws`.
- Frames (JSON text): `{kind: 'event', event}` and `{kind: 'control', type, ...}` (`stream.open`, `subscriptions`, `pong`, `error`).
- Client commands (validated): `subscribe` / `unsubscribe` (topics 1–50) / `ping` → `pong`.
- Rate limiting: fixed 60 s window, 120 messages/min default → `close(1008, 'rate limit exceeded')`.
- Heartbeat sweep: ping each client every `heartbeatMs`; clients that failed to pong are terminated (reaps half-open sockets behind NAT).
- Connection-limit rejection codes: 1013 (limit), 1011 (other).

7.6 `src/auth/jwt.ts` — ticket verification (131 lines)

- Only the **public half** of the RS256 pair lives in this process (`createPublicKey`) — a compromised realtime tier cannot forge tokens.
- `jwtVerify(jose)` with `algorithms: ['RS256'], issuer: incidentflow-api, audience: incidentflow-realtime, clockTolerance: 30`. The audience check is what stops a stolen REST access token from being replayed as a stream ticket — important because EventSource forces the credential into a query string.
- Error classification → `missing` / `expired` / `invalid_signature` / `invalid_claims` / `malformed`, each counted on `authFailures`.
- Claims validated with `TicketClaimsSchema`: `sub`, `org_id`, `role`, `name?`, `jti?`, `exp`.
- `extractCredential(headers, query)` — three places a browser can put a credential: `Authorization: Bearer` (preferred — out of access logs), `?ticket=` (EventSource), and the WebSocket subprotocol `["incidentflow.v1", "ticket.<jwt>"]`.

7.7 `src/app.ts`, `index.ts`, `middleware.ts`, `metrics.ts`, `logger.ts`, `types.ts`

`app.ts` (168 lines)

- `trust proxy` on; `x-powered-by` and `etag` off.
- Middleware chain: `requestId()` → `securityHeaders()` → `cors()` → `requestLogger()`.
- Endpoints: `GET /healthz` (liveness, dependency-free — a Redis outage must not trigger restarts of every pod), `GET /readyz` (checks Redis + draining flag → 503 when not ready), `GET /metrics` (Prometheus text), `GET /stream` (SSE).
- SSE handler flow: reject 503 when draining → extract credential → verify ticket → parse `topics` (max 50) → register (limits) → start → **Last-Event-ID replay or stream.gap** → teardown on close.
- 404 catch-all → `HttpError(404, 'not_found')`; terminal error handler emits the Laravel-compatible envelope `{error:{code, message, details, request_id}}`, masking messages in production.

`index.ts` (151 lines)

- Wiring: a `RedisChannelSubscriber` and a `Hub` reference each other via a holder object; the subscriber's `onEvent` dispatches into the hub and logs `event.dispatched` with delivered count.
- Tuned HTTP timeouts: `keepAliveTimeout 65_000`, `headersTimeout 70_000`, `requestTimeout 0` — long-lived SSE streams must never be reaped.
- **Shutdown sequence** (SIGTERM/SIGINT): force-exit timer armed for `SHUTDOWN_TIMEOUT_MS` → `hub.broadcastControl('stream.reconnect')` → sleep ≤ 1 s so the load balancer observes the failing readiness probe → close websockets (1001), close all, `server.close()`, `subscriber.quit()`, exit 0.

`middleware.ts` (158 lines)

`requestId`: accepts `x-request-id` (max 128 chars, else `randomUUID()`), echoes it — the same id flows nginx → Laravel → Redis envelope → realtime → response, so one grep reconstructs a user action across four processes. `requestLogger`: child logger with `request_id`; observes `httpRequestDuration` using `req.route?.path` (never raw URLs — label cardinality). `cors`: dev-only, echoes exact origins, allows Authorization/Content-Type/Last-Event-ID/X-Request-Id. `securityHeaders`: nosniff, DENY framing, no-referrer.

`metrics.ts` — Prometheus, deliberately small (no per-user/per-incident labels → no cardinality explosions)

```
incidentflow_realtime_connections / _connections_total / _connections_rejected_total (transport, reason)
incidentflow_realtime_subscriptions (active (connection, topic) pairs)
incidentflow_realtime_redis_channels
incidentflow_realtime_events_received_total (outcome:
accepted|unparseable|invalid|unsupported_version)
incidentflow_realtime_events_delivered_total / _delivery_failures_total
incidentflow_realtime_auth_failures_total (reason)
incidentflow_realtime_http_request_duration_seconds (histogram)
```

`logger.ts`

pino structured JSON to stdout, ISO timestamps, redaction paths (`req.headers.authorization`, `req.headers.cookie`, `token`, `ticket`, `*.password`, `*.refresh_token`), no pretty-printing — byte-identical dev/prod shapes.

`types.ts` — the wire contract

`ENVELOPE_VERSION = 1`; `RealtimeEventSchema` (version, id, type, organization_id, incident_id?, occurred_at, actor?, request_id, payload); `TicketClaimsSchema`; `Topic = `org:${number}` | `incident:${number}``. Topics are **filters, not access grants** — every event arrives on a per-org Redis channel the hub only subscribes to on behalf of an authenticated member of that org.

7.8 Tests and Docker image

`test/hub.test.ts` (252 lines, vitest) — 12 hub tests using a `FakeSubscriber` (records subscribe/unsubscribe) and a `FakeConnection` (records deliveries, supports `failNextDelivery`): reference counting, org fan-out,

boundary isolation, topic narrowing, org-topic unsubscribed refusal, per-user vs global limits, slow-consumer drop, replay after cursor, gap on aged-out cursor, activeChannels for reconnect restore, closeAll. Plus `isValidTopic` boundary cases. (13 tests total per README.)

Dockerfile — two stages:

```
FROM node:22-alpine AS build
COPY package.json package-lock.json ./ ; RUN npm ci --no-audit --no-fund
COPY tsconfig*.json ./ ; COPY src ./src ; RUN npm run build
RUN npm ci --omit=dev --no-audit --no-fund && npm cache clean --force

FROM node:22-alpine AS runtime
apk add --no-cache tini
ENV NODE_ENV=production
COPY --from=build --chown=node:node node_modules/ dist/ package.json ./
USER node ; EXPOSE 3001
HEALTHCHECK: node -e "fetch('http://127.0.0.1:$(process.env.PORT||3001)/healthz')"
ENTRYPOINT ["/sbin/tini", "--"] ; CMD ["node", "dist/index.js"]
```

tini does double duty: zombie reaping and SIGTERM forwarding, so the graceful shutdown sequence in `index.ts` actually runs.

8. Part 3 — Web SPA (React 19 + TypeScript + Vite)

8.1 Scaffold & dependencies

```
$ cd ..
$ npm create vite@latest web -- --template react-ts
$ cd web
$ npm install @tanstack/react-query:"^5.90.5" clsx:"^2.1.1" date-fns:"^4.1.0" lucide-react:"^0.545.0"
react:"^19.2.0" react-dom:"^19.2.0" react-hook-form:"^7.65.0" react-router-dom:"^7.9.5" recharts:"^3.2.1"
zod:"^4.1.14"
$ npm install -D @playwright/test:"^1.56.1" @tailwindcss/vite:"^4.1.16" tailwindcss:"^4.1.16" @testing-
library/jest-dom:"^6.9.1" @testing-library/react:"^16.3.0" @testing-library/user-event:"^14.6.1"
@types/node:"^24.10.1" @types/react:"^19.2.2" @types/react-dom:"^19.2.1" @vitejs/plugin-react:"^5.1.0"
eslint:"^9.39.1" eslint-plugin-react-hooks:"^7.0.1" jsdom:"^28.0.1" typescript:"^5.9.3" typescript-eslint:"^8.47.0"
vite:"^7.1.12" vitest:"^3.2.4"
```

Scripts:

```
"dev": "vite",
"build": "tsc -b && vite build",
"preview": "vite preview",
"typecheck": "tsc -b --noEmit",
"lint": "eslint .",
"test": "vitest run",
"test:watch": "vitest",
"e2e": "playwright test"
```

8.2 vite.config.ts and TypeScript

```
plugins: [react(), tailwindcss()], alias: { '@': './src' },

server: { port: 5173, strictPort: true, proxy: {
  '/api': { target: process.env.VITE_API_PROXY ?? 'http://localhost:8000', changeOrigin: true },
  '/realtime': {
    target: process.env.VITE_REALTIME_PROXY ?? 'http://localhost:3001',
    rewrite: p => p.replace(/^\/realtime/, ''), // /realtime/stream → /stream
    proxyRes: (res) => { res.headers['cache-control'] = 'no-cache, no-transform';
      res.headers['x-accel-buffering'] = 'no'; } // SSE must never buffer
  },
  '/ws': { target: 'http://localhost:3001', ws: true } // WebSocket proxy
}},

build: { outDir: 'dist', sourcemap: true, manualChunks: {
  react: ['react', 'react-dom', 'react-router-dom'],
  query: ['@tanstack/react-query'],
  charts: ['recharts'] // hundreds of KB out of the main bundle
}},

test: { environment: 'jsdom', globals: true,
  setupFiles: ['./src/test/setup.ts'], css: false,
  exclude: ['node_modules', 'dist', 'e2e/**'] } // Playwright specs ≠ vitest
```

The dev proxy mirrors production's single origin — so the refresh cookie, CORS behaviour and SSE proxying are exercised in development rather than discovered at deploy time.

`tsconfig.json` is the standard Vite split (`tsconfig.app.json` for `src`, `tsconfig.node.json` for config files) with `strict` and the `@` path alias.

8.3 src/lib/ — the door to the API

`api-client.ts` (360 lines) — the single door

- `ApiError(status, code, message, details, requestId); isTransient() =  $\geq 500$  or 429;` `fieldErrors()` flattens `details.fields` for form rendering.
- **Token storage:** module-level closure variable — *not* `localStorage`, not `sessionStorage` ("anything an XSS payload can read, it will exfiltrate; a closure variable dies with the tab"). The refresh token lives in an `HttpOnly` cookie the page cannot read.
- `refreshAccessToken()`: a **single shared in-flight promise** — when an access token expires every query fails at once; ten concurrent refreshes would present an already-rotated token to the server → reuse detection → family revocation. The slot clears in a `queueMicrotask`.
- `request()`: `API_BASE = '/api/v1'`; query serialization (arrays → `key[]=`, booleans → `'1'/'0'` — Laravel's boolean rule rejects `"true"`); headers `Accept`, `X-Request-Id` (fresh uuid per logical operation), `Authorization`, `X-Organization`.
- **Idempotency:** default `true` for `POST` — the key is *derived from the stable request id*, so a refresh-retry of the same operation carries the same key: "an expired token mid-create cannot produce two incidents".
- **401 refresh-retry:** only for `token_expired/unauthenticated` codes ("a revoked or malformed one will not become valid by asking again"), retried once.

- Responses validated with zod at the boundary; schema mismatch → `schema_mismatch` `ApiError` ("a contract mismatch is a bug, not a user error").
- `download(path)`: authenticated file download — `<a href>` can't carry the bearer token and a query-string token would land in history + access logs; fetches to a blob, synthetic click, revokes the object URL.

`query-client.ts`

`staleTime: 30_000` (SSE delivers live updates, so aggressive polling is unnecessary), `gcTime: 5 min`, refetch on window focus/reconnect. Retry policy: non-transient `ApiError` → no retry; 429 → at most once; others → max 2, exponential 1 s → 8 s. **Mutations are never retried** — "the user pressed a button once; silently pressing it again on their behalf is how a single 'Resolve' becomes two timeline events."

`schemas.ts` (316 lines)

zod runtime contracts mirroring the API resources: `ErrorEnvelopeSchema`, `User/Organization/Service`, `IncidentSchema` (severity/status as objects with `allowed_transitions` — "the UI can never offer a transition the state machine will reject"), `IncidentEvent/Comment/Update/Postmortem/Notification/AuditLog`, `DurationStats/MetricsSummary/TrendPoint`, `MemberSchema` (role, role_label, permissions), `RealtimeTicketSchema`, `RealtimeEnvelopeSchema`, plus `paginated()` (handles page- and cursor-based pagination) and `wrapped()`.

`format.ts` + `format.test.ts`

`formatDuration` — progressive precision (45s → 59m → 1h 30m → 1d 1h; **null/negative** → '—': a null MTTR means "never resolved", emphatically not "0s"). `formatRelative` (date-fns strict + "ago"), `initials`, `titleCase` (`incident.status_changed` → "Incident Status Changed"), `formatPercent`, `formatNumber` (Intl).

8.4 `src/auth/` — session management

`AuthProvider.tsx` (246 lines)

- Context: `status` (`loading|authenticated|anonymous`), `user`, `memberships`, `organization`, `permissions: Set<string>`, `can()`, `login/register/logout/switchOrganization`.
- **Session restore on load**: calls `refreshAccessToken()`; no token → anonymous; token → `loadProfile()` (GET `/auth/me`), picks the active org — remembered slug from `localStorage['incidentflow.organization']` if still a member, else the first membership (so removal from an org doesn't strand the user).
- **Proactive refresh** at 80% of token life (`max(30s, expiresIn*1000*0.8)`) — keeps the credential warm so the first action after an idle period doesn't pay a refresh round-trip during an incident. The loop lives in a ref so the self-referencing callback can re-arm without closing over a stale render.
- **logout**: POST `/auth/logout` (204), but a failed logout still clears local state — "leaving the user apparently signed in because the network blipped is the worse outcome".
- **switchOrganization(slug)**: sets the slug then `window.location.reload()` — "a full reload is the honest way to drop every cached query belonging to the previous tenant".

`useAuth.ts` — the 12-line context hook, throws outside the provider.

8.5 `src/realtime/useIncidentStream.ts` (221 lines) — the entire reason the design works

Three documented non-obvious points:

1. **The ticket outlives nothing** — stream credentials are valid for 60 s. `EventSource` reconnects on its own, but to the *same* URL with the now-expired ticket, forever. So the `error` handler closes the stream and re-opens with a freshly minted ticket; the browser's built-in retry is deliberately not relied upon.
2. **A gap is reported, not hidden** — on `stream.gap` the hook does `queryClient.invalidateQueries()`: refetch everything from PostgreSQL, the source of truth.
3. **Events invalidate, they do not patch** — the envelope carries what changed, not the full resource; splicing partial data would let the cache drift from the server's version.

Flow: `POST /realtime/ticket` (idempotent: false) → topics = `ticket.topics` + optional `incident:{id}` → URL from `ticket.stream_url` with `?ticket=` and `?topics=`; manual reconnects append `?last_event_id=` (`EventSource` can't set headers, so the resume cursor goes in the query string). Control frames: `stream.open` → live; `stream.gap` → full invalidation; `stream.reconnect` → close + schedule reconnect (node draining — move promptly instead of waiting for the socket to die). Every one of the 18 known event types invalidates `incidentKeys.lists()`, `['metrics']`, `['notifications']`, and — when an `incident_id` is present — that incident's detail/timeline/comments.

Reconnect logic: exponential backoff with jitter — `base = min(30s, 1000 · 2min(attempt, 5))`, `delay = base/2 + random · base/2`. Without jitter, every browser connected to a restarted node returns in the same millisecond (thundering herd).

Status: `connecting` | `live` | `reconnecting` | `offline`.

8.6 `src/hooks/` — React Query data

- `queryKeys.ts` — a key factory: centralised so invalidation is reliable (`incidentKeys.lists()` must invalidate every list regardless of filters, which only works if every list shares a prefix).
- `useIncidents.ts` (292 lines) — list with `placeholderData: previous` (paging never blanks the table); `useTransitionIncident` with optimistic update: snapshot → patch cached status (and `empty allowed_transitions` — "the client does not own the state machine, so it must not guess which buttons come next") → `onError` restore the exact snapshot → `onSettled` invalidate. `useAddComment` — optimistic comment with a *negative id* (`-Date.now()`) so reconciliation can never collide with a server id.
- `useMembers.ts` — `assignableOnly` sent to the server rather than filtered client-side, so the two definitions of "assignable" can never drift.
- `usePostmortem.ts` — a 404 is an expected answer (no postmortem yet): caught and returned as `null`; save is **PUT** (one per incident; saving the same draft twice must be one document); publish seeds the cache so `missing_sections` updates the button without a second round-trip.

8.7 `src/components/`

Component	Highlights
AppLayout	Sticky blurred header: logo, org selector (only when multiple memberships), stream status indicator , avatar, sign-out. Permission-filtered nav. Opens the single session-wide <code>useIncidentStream()</code> so navigation never tears down the connection. The indicator is

	<code>role="status"/aria-live="polite"</code> — green "Live" vs amber "Reconnecting...", explicitly not decoration.
NewIncidentDialog	Native <code><dialog></code> + <code>showModal()</code> (free focus trap, Escape-to-close, ARIA). On success navigates before closing — <code>onClose()</code> unmounts and a <code>navigate()</code> from an unmounted component is dropped, leaving the reporter unsure whether their SEV-1 was filed.
IncidentControls	SeverityControl (commander-only; a downgrade requires a 10-char reason collected before submit — the rule itself lives on the API, this is courtesy not enforcement); ResponderControl (assignableOnly picker); UpdatesPanel (the communications log with "Safe for a customer status page" flag).
Timeline	Rendered as an <code></code> ("a screen reader user should hear '3 of 11'"), colour-coded dots per event type, <code><time></code> with title, and the mono request_id on entries likely to be investigated — "an engineer can jump straight from the timeline to the matching server logs".
PostmortemEditor	Seven sections defined once; local state holds only touched fields, everything else derived from server copy at render time (no dual source of truth to drift). Published = read-only (corrections are amendments, not silent rewrites). Publish button disabled (not hidden) with missing sections spelled out.
Badges	SEV-1 gets a heavier 2px ring + semibold as a <i>shape</i> cue — "one in twelve men has some form of colour vision deficiency... colour is never the only signal". Status badge pulses for open/acknowledged; pulse suppressed under prefers-reduced-motion .
EmptyState / ErrorBoundary / FullPageSpinner	ErrorBoundary is the last line of defence against a white screen: surfaces the error, shows the request id for ApiErrors, offers Try again / Reload.

8.8 [src/pages/](#)

Page	Role
Login	Passes the server's message straight through (the API deliberately returns the same message for unknown email and wrong password; re-mapping would reintroduce the enumeration leak). Collapsible "Demo accounts" list.
Register	Fields name/organization_name/email/password(+confirmation); 422 errors against the right field via <code>fieldErrors()</code> ; notes the 12-character minimum ("length beats punctuation").
Incidents	Filterable table (search, status, severity, active-only; any change resets to page 1), 25/page, permission-gated toolbar (CSV export honours the filters on screen — WYSIWYG; red "Report incident" button), skeleton while loading.
IncidentDetail	Header + TransitionBar rendered from the server's allowed_transitions ; left column timeline/updates/postmortem/discussion; right column metadata dl, responder roster, commander-only severity control. Deliberately opens no stream of its own — the layout's session-wide stream already invalidates the right queries.
Postmortems	List with status labels and a highlight of <code>missing_sections</code> ("naming what's missing turns a blocked publish into a to-do list").

Metrics	7/30/90-day toggle; four stat cards (opened, open now, MTTA, MTTR — averages always shown with percentiles); SLA attainment per severity; recharts panels "Opened vs resolved" and "Daily mean time to resolve" with <code>connectNulls</code> — days with no resolutions must not draw a zero that claims an instant recovery.
Services	Card grid with tier labels and a red "N open" chip showing <i>active</i> incident count ("a service that had a rough quarter but is healthy today does not look on fire").
Admin	Members table + audit log (append-only, permanent — "an audit log an administrator can rewrite proves nothing about administrators").
NotFound	Minimal 404.

8.9 App, routing, styling

`main.tsx`: `StrictMode` → `QueryClientProvider` → `BrowserRouter` → `AuthProvider` (inside the router – switching organizations triggers navigation – and inside the query client – restoring a session invalidates cached data) → `<App/>`.

`App.tsx`: `status` `===` `'loading'` → `FullPageSpinner` ("Restoring your session..." — never flash login at an authenticated user on refresh). Anonymous: `/login`, `/register`, everything else → redirect. Authenticated (all inside `<AppLayout/>`, wrapped in `ErrorBoundary`): `index` → `/incidents`; `/incidents`, `/incidents/:id`, `/services`, `/metrics`, `/postmortems`, `/admin`; `/login` → redirect (already-authenticated guard); `*` → `NotFound`.

`index.css` : Tailwind 4 via `@import "tailwindcss"`; a dark slate/red palette; the global `prefers-reduced-motion` rule for pulse dots; system font stack.

8.10 Unit tests, e2e tests, Docker image

`src/test/setup.ts`: jest-dom matchers. Unit tests: `format.test.ts` (duration boundaries, initials, titleCase).

`e2e/incident-lifecycle.spec.ts` — Playwright, 7 tests, one browser (chromium)

1. Responder signs in and sees seeded incidents.
2. Session survives a reload — proves the `HttpOnly` refresh-cookie path end-to-end.
3. Reporter opens an incident and it appears on the list (expects `INC-...` reference and timeline entry).
4. Responder drives an incident through its lifecycle (clicks whatever `allowed_transitions` offers; asserts no "Mark open" button on an open incident).
5. Viewer cannot report an incident (control absent because the permission is absent).
6. **The realtime stream connects through nginx** — "the single most valuable assertion": the Live indicator must appear within 15 s, which only passes if ticket minting, unbuffered proxying, signature verification and channel prefixes all line up.
7. Metrics render from real seeded history.

`playwright.config.ts` : `testDir` `'./e2e'`, `workers` `1`, `retries` `1` in CI, `baseUrl` from `E2E_BASE_URL` (default `http://localhost:8080`), `expect.timeout` 10 s (raise to 45 s against `php artisan serve`), trace on first retry, screenshots on failure, video retain-on-failure.

Dockerfile — two stages

```
FROM node:22-alpine AS build
COPY package.json package-lock.json* ./ ; RUN npm ci --no-audit --no-fund
COPY . .
ARG VITE_APP_VERSION=dev / ENV VITE_APP_VERSION=$VITE_APP_VERSION # Vite inlines at build time
RUN npm run build

FROM nginx:1.27-alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
COPY docker/nginx-spa.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK: wget -q --spider http://127.0.0.1/ || exit 1
```

`docker/nginx-spa.conf` — SPA fallback `try_files $uri $uri/ /index.html;; /assets/` cached public, immutable 1 year; `index.html` itself `no-store` (so deploys land instantly); own gzip block.

9. Part 4 — infra/nginx (the edge)

Two files: `infra/nginx/nginx.conf` (main process) and `infra/nginx/default.conf` (server block). **The API is served via FastCGI to PHP-FPM** (not `proxy_pass`), the realtime tier via `proxy_pass`, the SPA via `proxy_pass` to its inner nginx. `infra/keys/` exists but stays empty — gitignored; real keys live in the `jwt-keys` Docker volume.

nginx.conf — highlights

```
# Structured JSON access log - ONE tool reads both nginx and app logs
log_format json_combined escape=json
'{ "time": "$time_iso8601", "remote_addr": "$remote_addr",
  "request_id": "$request_id_final", "method": "$request_method",
  "uri": "$request_uri", "status": $status, "bytes": $body_bytes_sent,
  "duration_ms": $request_time_ms, "upstream_ms": "$upstream_response_time",
  "referrer": "$http_referer", "user_agent": "$http_user_agent" }';

# The correlation id is MINTED HERE, honouring a caller-supplied one
map $http_x_request_id $request_id_final { default $http_x_request_id; "" $request_id; }

# WebSocket upgrade mapping
map $http_upgrade $connection_upgrade { default upgrade; '' close; }

sendfile on; tcp_nopush on; tcp_nodelay on; server_tokens off;
keepalive_timeout 65; client_max_body_size 12M;
gzip on; gzip_vary on; gzip_proxied any; gzip_min_length 1024;
gzip_types text/plain text/css application/json application/javascript
        application/xml image/svg+xml;

# Edge rate limiting, BEFORE PHP starts
limit_req_zone $binary_remote_addr zone=api_general:16m rate=30r/s;
limit_req_zone $binary_remote_addr zone=api_auth:8m rate=1r/s;
limit_conn_zone $binary_remote_addr zone=per_ip_conn:16m;
```

default.conf — highlights

```
upstream api_fpm          { server api:9000; keepalive 16; }
upstream realtime_service { server realtime:3001; keepalive 32; }
upstream web_spa          { server web:80; keepalive 8; }

# Security headers (always)
X-Content-Type-Options nosniff | X-Frame-Options DENY | Referrer-Policy strict-origin-when-cross-
origin
Permissions-Policy camera=(), microphone=(), geolocation=(), interest-cohort=()
Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';
  img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self';
  frame-ancestors 'none'; base-uri 'self'; form-action 'self'"
limit_conn per_ip_conn 64;

location /realtime/ {                                # SSE - never buffer
  proxy_pass http://realtime_service/;
  proxy_http_version 1.1; proxy_buffering off; proxy_cache off; gzip off;
  proxy_set_header Connection '';
  proxy_set_header X-Request-Id $request_id_final;
  proxy_read_timeout 3600s; proxy_send_timeout 3600s;
  add_header X-Accel-Buffering "no" always;
}
location /ws {                                         # WebSocket upgrade
  proxy_pass http://realtime_service;
  proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade;
  proxy_set_header Connection $connection_upgrade;
  proxy_read_timeout 3600s; proxy_buffering off;
}
location ~ ^/api/v1/auth/(login|register|refresh)$ {  # tighter auth bucket
  limit_req zone=api_auth burst=5 nodelay;
  try_files $uri /index.php?$query_string;
}
location /api/ {
  limit_req zone=api_general burst=40 nodelay;
  try_files $uri /index.php?$query_string;
}
location = /api/v1/health/live { try_files $uri /index.php?$query_string; } # rate-limit exempt
location = /api/v1/health/ready { try_files $uri /index.php?$query_string; }
location /horizon { try_files $uri /index.php?$query_string; }
location ~ ^/index\.php(/|$) {                       # the FastCGI block - internal
  fastcgi_pass api_fpm;
  fastcgi_param HTTP_X_REQUEST_ID $request_id_final;
  fastcgi_hide_header X-Powered-By;
  fastcgi_keep_conn on;                               # streaming CSV reaches the client as produced
}
location ~ \.php$ { return 404; }                     # nothing else executes PHP (RCE guard)
location ~ /\.(!well-known) { deny all; access_log off; log_not_found off; }
location / { proxy_pass http://web_spa; ... }         # the SPA
```

10. Part 5 — Docker Compose (dev + prod)

`docker-compose.yml` — the dev stack

```
name: incidentflow

x-api-environment: &api-environment
  APP_NAME: IncidentFlow, APP_ENV: local, APP_DEBUG: "true"
  APP_KEY: ${APP_KEY:-base64:0000 ...} # dev default; prod REQUIRES it
  APP_URL / APP_FRONTEND_URL: http://localhost:8080
  LOG_CHANNEL: stack, LOG_STACK: json, LOG_LEVEL: debug
  DB_CONNECTION: pgsql, DB_HOST: postgres, DB_DATABASE/USERNAME/PASSWORD: incidentflow
  REDIS_CLIENT: predis, REDIS_HOST: redis, REDIS_PORT: 6379
  CACHE_STORE: redis, QUEUE_CONNECTION: redis, SESSION_DRIVER: redis
  REALTIME_ENABLED: "true", REALTIME_CHANNEL_PREFIX: incidentflow
  MAIL_MAILER: smtp, MAIL_HOST: mailpit, MAIL_PORT: 1025
  JWT_REFRESH_COOKIE_SECURE: "false", JWT_REFRESH_COOKIE_SAMESITE: lax

services:
  postgres: postgres:16-alpine, POSTGRES_INITDB_ARGS "--encoding=UTF8 --locale=C",
    port 5432 published, healthcheck pg_isready (named user+db explicitly)
  redis: redis:7-alpine, "redis-server --appendonly no --maxmemory 256mb
    --maxmemory-policy allkeys-lru", port 6379, healthcheck redis-cli ping
    # everything here is cache/queue/pubsub – PostgreSQL is the truth
  mailpit: axllent/mailpit:latest, port 8025 (catches every outgoing email)
  api: build ./api target runtime; bind-mount ./api:/var/www/html,
    api-vendor + jwt-keys volumes; depends_on postgres+redis healthy
  migrate: one-shot: "sh -c 'php artisan migrate --force && php artisan db:seed --force'",
    restart "no" – keeps replicas from racing through schema changes
  horizon: ["php", "artisan", "horizon"], stop_grace_period 30s
  scheduler: ["php", "artisan", "schedule:work"] # schedule lives in routes/console.php
  realtime: build ./realtime target runtime; env: PORT 3001, REDIS_URL redis://redis:6379,
    JWT_PUBLIC_KEY_PATH /keys/jwt-public.pem (the PUBLIC half only),
    JWT_ISSUER incidentflow-api, JWT_AUDIENCE incidentflow-realtime,
    CORS_ORIGINS http://localhost:8080,http://localhost:5173;
    volume jwt-keys:/keys:ro
  web: build ./web target runtime
  nginx: nginx:1.27-alpine, port 8080:80, mounts nginx.conf + default.conf (ro)
    AND ./api:/var/www/html:ro (nginx serves public/index.php via FastCGI),
    healthcheck wget /api/v1/health/live

volumes: postgres-data, api-vendor, jwt-keys
```

`docker-compose.prod.yml` — the production overlay

Applied on top: `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`. It changes four things that matter:

1. **No bind mounts** — the image is the artifact; a container that reads code from the host is not the thing that was tested in CI.
2. **No published database/Redis/mail ports** — only nginx is reachable (`ports: !override []`).
3. **Secrets from the environment, never defaults** — `${APP_KEY:?APP_KEY must be set}` etc.; Redis runs with `--requirepass ${REDIS_PASSWORD:?}`; mailpit gets `profiles: ["never"]`.

4. **Resource limits and restart policies** — API 2 replicas (512 MB, 1 CPU), realtime 2 (384 MB), web 2 (128 MB), horizon 1, scheduler 1 (exactly one).

Also: images from `${REGISTRY:-ghcr.io/incidentflow}/{api,realtime,web}:${APP_VERSION:-latest}`; migrate runs migrations only (seeding demo accounts into production would be a data breach with extra steps — DatabaseSeeder refuses there anyway); refresh cookie `Secure=true/SameSite=strict`; realtime `CORS_ORIGINS=${APP_FRONTEND_URL}`, `SHUTDOWN_TIMEOUT_MS=15000`, `stop_grace_period 30 s`.

11. Part 6 — GitHub Actions CI/CD

One workflow, `.github/workflows/ci.yml` (310 lines): on push to main / pull_request / workflow_dispatch; concurrency group per ref with cancel-in-progress. Global env `PHP_VERSION: 8.3`, `NODE_VERSION: 22`. Five jobs:

Job	Steps (commands you would run locally)
api (15 min; services: postgres:16-alpine + redis:7-alpine)	<code>setup-php 8.3</code> (extensions pdo_pgsql, mbstring, openssl, bcmath, pcntl, posix, zip; composer v2) → <code>composer install --prefer-dist --no-interaction --no-progress → cp .env.example .env; php artisan key:generate; php artisan jwt:keys --force → composer run lint</code> (pint --test) → <code>php vendor/bin/phpunit --testdox</code> (SQLite) → phpunit again with <code>DB_CONNECTION=pgsql CACHE_STORE=redis</code> (real PostgreSQL) → migration rollback check : <code>migrate --force → migrate:rollback --force --step=100 → migrate --force</code> ("a migration nobody can reverse is a deployment with no way back") → <code>migrate:fresh --seed --force → composer audit</code>
realtime (10 min)	<code>npm ci --no-audit --no-fund → npm run typecheck → npm run lint → npm test → npm run build → npm audit --audit-level=high</code>
web (10 min)	same chain: <code>npm ci → typecheck → lint → test → build → audit</code>
images (25 min, needs the three above; matrix over api/realtime/web)	<code>buildx → login to ghcr.io</code> (push only) → <code>build-push-action</code> , context <code>./<service></code> , target: runtime , push on push, tags <code>ghcr.io/<repo>/<svc>:<sha> + :latest</code> , build-args <code>APP_VERSION/VITE_APP_VERSION = git SHA</code> , gha cache per service → Trivy scan (SARIF, CRITICAL/HIGH, ignore-unfixed — "unfixed findings cannot be acted on today") → upload SARIF to code scanning
e2e (20 min, needs the three above)	hardcoded dev <code>APP_KEY</code> env → <code>docker compose up -d --build --wait --wait-timeout 300 → npm ci</code> in web, <code>npx playwright install --with-deps chromium → E2E_BASE_URL=http://localhost:8080 npm run e2e</code> → on failure: <code>dump docker compose logs --no-color --tail 200</code> , upload web/test-results as artifact, always <code>docker compose down -v</code>

12. Part 7 — Makefile & documentation

The `Makefile` is deliberately thin — "nobody should have to read this file to know what a target does to their machine". Targets: `up` (`docker compose up --build`), `down` (`down -v`), `logs`, `shell`, `test` (api+realtime+web), `test-api/test-realtime/test-web`, `e2e`, `lint`, `build` (prod compose build), `migrate`, `seed`, `fresh`, `prune` (dry-run), `keys` (jwt:keys + restart of the four services), `clean`.

Documentation (all written as the project matured):

File	Content
README.md	The pitch, architecture diagram, division-of-responsibility table, run instructions, demo accounts, verification commands, implemented-feature list, docs index, MIT licence.
docs/architecture.md	Why the system is shaped this way: modular SOA not microservices, the realtime split, the 8-step data flow, invalidate-not-patch, Redis pub/sub rationale, per-org channels, three credentials, no global tenant scope, append-only by construction, metrics without rollup, rejected alternatives.
docs/api.md	Conventions (versioning, auth, error codes, rate limits, pagination, idempotency, correlation ids, CSV export), full route behaviour, the error-code table (~17 codes).
docs/openapi.yaml	OpenAPI 3.1.0, relative server <code>/api/v1</code> , 16 paths, security schemes (bearer JWT), reusable responses and 8 schemas.
docs/runbook.md	Deploying, zero-downtime migration constraints, rolling back, health probes, incident debugging via request ids, key rotation, pruning, backups, scaling, security notes.
docs/interview-notes.md	Six load-bearing decisions defended in depth, each with "what this costs".
docs/adr/0001-separate-realtime-service.md (2026-01-05)	Why fan-out moved to Node; boundary; consequences; alternatives rejected (polling, Pusher/Soketi, Swoole/FrankenPHP).
docs/adr/0002-no-global-tenant-scope.md (2026-01-06)	Why every query names its organization; the fails-open failure mode of global scopes.
.claude/launch.json	Dev launch config running <code>npm run dev</code> in <code>web/</code> on port 5173.

13. Running the whole stack

Option A — everything in Docker (recommended)

```
$ docker compose up --build
```

Open `http://localhost:8080`. Mail is captured by Mailpit at `http://localhost:8025`; the queue dashboard is at `/horizon`. Migrations and demo data run automatically via the one-shot `migrate` service. The header stream indicator switches from amber "Reconnecting..." to green "Live" once the pipeline works end to end.

Option B — without Docker (SQLite, no live updates)

```
$ cd api
```

```
$ cp .env.local.example .env
```

```
$ composer install
```

```
$ php artisan key:generate
```

```
$ php artisan jwt:keys
```

```
$ php artisan migrate --seed
```

```
$ php artisan serve
```

In a second terminal:

```
$ cd web
```

```
$ npm install
```

```
$ npm run dev
```

Live updates are disabled in this mode (`REALTIME_ENABLED=false`); the SPA falls back to refetching. Everything else works.

Option C — without Docker, with live updates (local Redis)

```
$ redis-server /path/to/redis.conf
```

```
$ cd api && php artisan queue:work redis
```

```
$ cd realtime
```

```
$ REDIS_URL=redis://127.0.0.1:6379 JWT_PUBLIC_KEY_PATH=./api/storage/keys/jwt-public.pem npm run dev
```

Uncomment the Redis block at the bottom of `api/.env.local.example` before starting the API. (Horizon needs `pcntl/posix` — Linux only; on Windows use `queue:work`.)

14. Demo accounts & what is implemented

Password `incidentflow` for all (seeded by `DatabaseSeeder`):

Email	Role	Can
<code>admin@incidentflow.test</code>	Administrator	Everything, including the audit log
<code>commander@incidentflow.test</code>	Incident Commander	Assign, change severity, publish postmortems
<code>responder@incidentflow.test</code>	Responder	Drive an incident through its lifecycle
<code>reporter@incidentflow.test</code>	Reporter	Open incidents, comment
<code>viewer@incidentflow.test</code>	Viewer	Read only

Core implemented: registration and login, incident creation with SEV-1...SEV-4, responder assignment, guarded status lifecycle, append-only timeline, internal comments, live updates, postmortems, MTTA/MTTR with percentiles, permanent audit log, queued email, CSV export, admin view.

Production concerns: RBAC, request validation (including query parameters), rate limiting, idempotency keys, pagination with a stable sort, optimistic UI updates, one central error shape, structured JSON logs, correlation IDs across all three services, liveness/readiness probes, soft deletes, transactions around every state change, queue retry with backoff, OpenAPI documentation, unit/feature/integration/E2E tests, dev and prod Docker configurations, GitHub Actions CI with image scanning, secrets kept out of the repository, seeded demo accounts.

15. Verifying everything (tests)

From a clean checkout:

```
cd api && composer test          # 103 tests, 505 assertions
cd realtime && npm test           # 13 tests
cd web && npm test && npm run build
```

End-to-end (drives a browser against the whole stack):

```
docker compose up -d --wait
cd web && npm run e2e          # 7 tests
```

E2E is written for the compose stack. Against `php artisan serve` the tests still pass, but that server handles one request at a time, so concurrent queries on a page queue behind each other — raise the assertion ceiling with `E2E_EXPECT_TIMEOUT=45000`.

16. Operating it — the runbook essentials

Deploying

```
export APP_VERSION=$(git rev-parse --short HEAD)
docker compose -f docker-compose.yml -f docker-compose.prod.yml build
docker compose -f docker-compose.yml -f docker-compose.prod.yml run --rm migrate
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Migrations run as a separate one-shot job, never in the container endpoint. Zero-downtime constraints: add columns nullable or with defaults (backfill later); never rename/drop in the same release that stops using it; index large tables concurrently.

Rolling back

```
APP_VERSION=<previous-sha> docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Roll the code back first; think about the schema separately (reversing a migration that accepted writes usually loses data).

Debugging with the request id

Every response carries `X-Request-Id`; the same value appears in nginx access logs, API logs, worker logs, inside the Redis envelope and on the timeline entry. One grep reconstructs a user action across four processes:

```
docker compose logs --no-color | grep '<request-id>'
```

Live updates stopped?

1. Is Redis reachable from both sides? (`docker compose exec redis redis-cli ping; curl realtime/readyz`)
2. **Do the channel prefixes match?** — the most common cause. The API's `REALTIME_CHANNEL_PREFIX` and the realtime service's `REDIS_CHANNEL_PREFIX` must be identical or publisher and subscriber sit on different channels with no error anywhere. Watch with `redis-cli PSUBSCRIBE 'incidentflow:*'`.
3. Is nginx buffering? `proxy_buffering off` on `/realtime/` is what keeps SSE alive.
4. Is the ticket being rejected? Check `incidentflow_realtime_auth_failures_total` on `/metrics`.

This is not an outage — PostgreSQL is the source of truth; the product works without the stream, just without live updates. Do not page for it out of hours.

Rotating the JWT key pair

```
docker compose exec api php artisan jwt:keys --force
docker compose restart api realtime horizon scheduler
```

Signs every session out immediately; the realtime service must be restarted too — it caches the public key at boot.

Backups

```
docker compose exec postgres pg_dump -U incidentflow incidentflow | gzip > backup.sql.gz
```

Restore-test this. "An untested backup is a belief, not a backup."

17. Appendix A — the complete command log

Everything typed into a terminal (or handed to an AI coding agent) to recreate this project from nothing:

```

# — 1. Repository —————
mkdir IncidentFlow && cd IncidentFlow
git init
# create: .editorconfig, .gitignore, Makefile, README.md (content in §5/§12)

# — 2. Laravel API —————
composer create-project laravel/laravel api "^12.0"
cd api
composer require laravel/horizon:"^5.30" predis/predis:"^2.2" firebase/php-jwt:"^6.11"
composer require --dev fakerphp/faker:"^1.23" laravel/pail:"^1.2.2" laravel/pint:"^1.18" \
    mockery/mockery:"^1.6" nunomaduro/collision:"^8.6" phpunit/phpunit:"^11.5.3"
# composer.json: scripts (lint/fix/test/check) + platform pins (see §6.1)

php artisan key:generate
php artisan jwt:keys # custom command (app/Console/Commands/GenerateJwtKeys.php)
php artisan vendor:publish --tag=horizon-config

# config: config/jwt.php, config/incidents.php, config/realtime.php,
#         edits to auth.php/database.php/queue.php/logging.php/app.php
# bootstrap/app.php: apiPrefix api/v1, middleware alias/prepend, exceptions (see §6.4)

php artisan make:middleware ForceJsonResponse
php artisan make:middleware AssignRequestId
php artisan make:middleware NormalizeBooleanParameters
php artisan make:middleware AuthenticateJwt
php artisan make:middleware EnsureOrganizationContext
php artisan make:middleware HandleIdempotency

# app/Enums: IncidentStatus, IncidentSeverity, IncidentEventType, Permission,
#            OrganizationRole, AssigneeRole, NotificationChannel,
#            NotificationStatus, PostmortemStatus

php artisan make:migration create_organizations_table
php artisan make:migration create_services_table
php artisan make:migration create_incidents_table
php artisan make:migration create_incident_activity_tables
php artisan make:migration create_platform_tables

php artisan make:model Organization -mfs
php artisan make:model OrganizationMember -mfs
php artisan make:model Service -mfs
php artisan make:model Incident -mfs
php artisan make:model IncidentEvent -mfs
php artisan make:model IncidentUpdate -mfs
php artisan make:model IncidentComment -mfs
php artisan make:model IncidentAssignee -mfs
php artisan make:model Postmortem -mfs
php artisan make:model Notification -mfs
php artisan make:model AuditLog -mfs
php artisan make:model RefreshToken -mfs
php artisan make:model IdempotencyKey -mfs
# + app/Models/Concerns/BelongsToOrganization.php (trait)

# app/Services: Auth/{KeyProvider,TokenService,IssuedToken,TokenClaims},
#              Incidents/{IncidentService,PendingEffects,TimelineRecorder},
#              Realtime/{RealtimeEvent,RealtimePublisher}, Notifications/NotificationDispatcher,
#              Metrics/MetricsService, Audit/AuditLogger, Support/RequestContext
# app/Auth: JwtGuard, UserProvider

php artisan make:policy IncidentPolicy --model=Incident
php artisan make:policy OrganizationPolicy --model=Organization

```

```

php artisan make:policy ServicePolicy --model=Service
php artisan make:policy PostmortemPolicy --model=Postmortem

# app/Http/Controllers/Api/V1: Health, Auth, Organization, Member, Service, Incident,
#   IncidentStatus, IncidentSeverity, IncidentCommander, IncidentAssignee,
#   IncidentEvent, IncidentUpdate, IncidentComment, IncidentExport,
#   Postmortem, Metrics, Notification, AuditLog, RealtimeTicket (19 controllers)
# app/Http/Requests/Auth + Incidents (Login, Register, StoreIncident,
#   UpdateIncident, TransitionIncident, IndexIncidents)
# app/Http/Resources: 11 resources
# app/Jobs/SendIncidentNotification.php, app/Mail/IncidentNotificationMail.php
#   + resources/views/mail/incident-notification.blade.php
# app/Console/Commands: GenerateJwtKeys, PruneExpiredRecords, RetryStaleNotifications
# app/Exceptions: DomainException, ApiExceptionRenderer, InvalidTokenException,
#   IncidentTransitionException, IdempotencyConflictException, PostmortemException
# routes/api.php (the full table, §6.16), routes/web.php, routes/console.php
# app/Providers/AppServiceProvider.php: singletons, auth extension, policies,
#   password policy, rate limiters, model strictness, URL scheme

php artisan migrate --seed
php artisan serve                      # dev without Docker

# tests: tests/TestCase.php + Unit/{OrganizationRoleTest,IncidentStatusTest}
#   + Feature/Auth, Feature/Incidents, Feature/Postmortems, ... (110 tests)
# phpunit.xml: sqlite :memory:, lowered bcrypt cost, JSON logging
# Dockerfile (3 stages), docker/{entrypoint.sh,php.ini,php-fpm.conf}

# — 3. Realtime service —————
cd ..
mkdir realtime && cd realtime
npm init -y
npm install express:"^5.1.0" ioredis:"^5.8.2" jose:"^6.1.0" pino:"^9.13.1" \
  prom-client:"^15.1.3" ws:"^8.18.3" zod:"^4.1.14"
npm install -D @types/express @types/node @types/supertest @types/ws eslint \
  ioredis-mock supertest tsx typescript typescript-eslint vitest
# tsconfig.json, tsconfig.build.json, eslint.config.js
# src: index, app, hub, redis, config, middleware, metrics, logger, types
#   transports/{sse,ws}, auth/jwt
# test/hub.test.ts
# Dockerfile (2 stages, tini entrypoint)

# — 4. Web SPA —————
cd ..
npm create vite@latest web -- --template react-ts
cd web
npm install @tanstack/react-query clsx date-fns lucide-react react react-dom \
  react-hook-form react-router-dom recharts zod
npm install -D @playwright/test @tailwindcss/vite tailwindcss @testing-library/jest-dom \
  @testing-library/react @testing-library/user-event @types/node @types/react \
  @types/react-dom @vitejs/plugin-react eslint eslint-plugin-react-hooks jsdom \
  typescript typescript-eslint vite vitest
# vite.config.ts (proxies + vitest), tsconfig*.json, eslint.config.js
# src: main, App, index.css; lib/{api-client,query-client,schemas,format(+test)};
#   auth/{AuthProvider,useAuth}; realtime/useIncidentStream; hooks/{useIncidents,
#   usePostmortem,useMembers,queryKeys}; components/{AppLayout,NewIncidentDialog,
#   IncidentControls,Timeline,PostmortemEditor,Badges,EmptyState,ErrorBoundary,
#   FullPageSpinner}; pages/{Login,Register,Incidents,IncidentDetail,Postmortems,
#   Metrics,Services,Admin,NotFound}
# e2e/incident-lifecycle.spec.ts, playwright.config.ts, src/test/setup.ts
# Dockerfile (2 stages), docker/nginx-spa.conf

```

```
# — 5. infra & compose —————
cd ..
mkdir infra/nginx infra/keys scripts docs/adr
# infra/nginx/{nginx.conf,default.conf}      (see §9)
# docker-compose.yml, docker-compose.prod.yml (see §10)

# — 6. CI —————
# .github/workflows/ci.yml (see §11)

# — 7. Documentation —————
# README.md, docs/{architecture,api,runbook,interview-notes}.md,
# docs/openapi.yaml, docs/adr/0001-*.md, docs/adr/0002-*.md

# — 8. Run & verify —————
docker compose up --build          # http://localhost:8080
cd api && composer test
cd realtime && npm test
cd web && npm test && npm run build
cd web && npm run e2e                # needs the compose stack up
```

18. Appendix B — file inventory

Complete source tree (excluding `node_modules`, `vendor`, `dist` build output and lockfiles):

```

IncidentFlow/
├── .editorconfig .gitignore Makefile README.md
├── docker-compose.yml docker-compose.prod.yml
├── .claude/launch.json
├── .github/workflows/ci.yml
├── api/
│   ├── Dockerfile phpunit.xml composer.json
│   ├── docker/entrypoint.sh php.ini php-fpm.conf
│   ├── bootstrap/app.php routes/{api,web,console}.php
│   ├── config/{app,auth,cache,database,filesystems,incidents,jwt,logging,
│   │   │   │   mail,queue,realtime,services,session}.php
│   ├── app/
│   │   ├── Auth/{JwtGuard,AuthProvider}.php
│   │   ├── Console/Commands/{GenerateJwtKeys,PruneExpiredRecords,RetryStaleNotifications}.php
│   │   ├── Enums/ (9 files)
│   │   ├── Exceptions/ (6 files)
│   │   ├── Http/Controllers/{Controller.php,Api/V1/*.php (19)}
│   │   ├── Http/Middleware/ (6 files)
│   │   ├── Http/Requests/{Auth/*,Incidents/*} (6 files)
│   │   ├── Http/Resources/ (11 files)
│   │   ├── Jobs/SendIncidentNotification.php
│   │   ├── Mail/IncidentNotificationMail.php
│   │   ├── Models/ (15 files + Concerns/BelongsToOrganization.php)
│   │   ├── Policies/ (4 files + Concerns/ChecksOrganizationPermissions.php)
│   │   ├── Providers/AppServiceProvider.php
│   │   ├── Services/{Auth(4),Audit,Incidents(3),Metrics,Notifications,Realtime(2),Support(2)}
│   │   └── Support/{RequestContext}.php
│   ├── database/migrations/ (8 files) seeders/ factories/ (8 files)
│   ├── resources/views/mail/incident-notification.blade.php
│   └── tests/ (12 files: 16 Unit + 94 Feature)
├── realtime/
│   ├── Dockerfile package.json tsconfig.json tsconfig.build.json eslint.config.js
│   ├── src/{index,app,hub,redis,config,middleware,metrics,logger,types}.ts
│   ├── src/transport/{sse,ws}.ts src/auth/jwt.ts
│   └── test/hub.test.ts
├── web/
│   ├── Dockerfile vite.config.ts playwright.config.ts eslint.config.js
│   ├── index.html tsconfig*.json
│   ├── docker/nginx-spa.conf
│   ├── e2e/incident-lifecycle.spec.ts
│   └── src/
│       ├── main.tsx App.tsx index.css
│       ├── auth/{AuthProvider.tsx,useAuth.ts}
│       ├── lib/{api-client,query-client,schemas,format(+test)}.ts
│       ├── realtime/useIncidentStream.ts
│       ├── hooks/{useIncidents,usePostmortem,useMembers,queryKeys}.ts
│       ├── components/ (9 files)
│       ├── pages/ (9 files)
│       └── test/setup.ts
├── infra/nginx/{nginx.conf,default.conf} infra/keys/ (empty, gitignored)
├── scripts/ (empty)
└── docs/ architecture.md api.md runbook.md interview-notes.md
    openapi.yaml adr/{0001,0002}*.md

```

This guide was produced by analyzing the repository at D:\IncidentFlow as-is; no files in the project were modified. Reference documents: README.md, docs/architecture.md, docs/api.md, docs/runbook.md, docs/interview-notes.md, docs/adr/*, and the source code itself.